

VPA IGCSE COMPUTER SCIENCE (0478)

— YEAR 9 SEMESTER 2 #

WEEKS 1–6: ALGORITHMS & PYTHON FUNDAMENTALS

Victoria Park Academy
Shenzhen #

EAL Context | Mixed Ability |
40-min Lessons #

COURSE OVERVIEW

This semester prepares students for CIE IGCSE Computer Science Paper 2

(Problem Solving & Programming, 50% of final assessment).

Semester 1 Recap Covered:

- Data Representation (binary, hex, sound, images)
- Data Transmission (packet switching, error detection, protocols)
 - Hardware (CPU, memory, I/O, storage)
- Software (system software, application software)
- Internet & Security (threats, prevention)
- Emerging Technology (AI, robotics)
- Python Intro: print(), variables, input(), basic I/O

Semester 2 Structure:

Weeks 1–3: Algorithms, Pseudocode & Flowcharts (Paper 2 Theory)

Weeks 4–10: Python Programming (Paper 2 Practical)

Weeks 11–12: Revision & End-of-Year Assessment

PREREQUISITES

- Y9S1 Python introduction (print, variables, input/output)
- Understanding of binary and how computers store data

- Basic computer literacy (file management, typing)

ASSESSMENT ALIGNMENT

Paper 2 Topics Covered This Semester:

- 2.1 Algorithm Design (pseudocode, flowcharts)
- 2.2 Data Types & Structures
- 2.3 Programming (Python fundamentals)
- 2.4 Selection (IF statements)
- 2.5 Iteration (loops — Weeks 7-8)
- 2.6 Arrays/lists (Week 9)
- 2.7 String handling (Week 9)
- 2.8 File handling (Week 10)
- 2.9 Validation & Testing (Week 10)
- 2.10 Testing strategies

DIFFERENTIATION FRAMEWORK

Emerging — Students who need significant scaffolding, sentence starters, bilingual glossaries, and additional teacher support.

Developing — Students working at expected level with standard tasks.

Secure — Students who grasp concepts quickly and need extension challenges, optimization problems, and independent projects.

EAL SUPPORT STRATEGIES (Used Throughout)

- Bilingual keyword lists (English / Chinese / Pinyin) in every lesson
- Visual anchors: diagrams, colour-coded syntax highlighting
- Sentence frames for pseudocode explanations
- Think-Pair-Share before whole-class responses
- Cognates highlighted (e.g., algorithm → 算法)
- Word banks on every worksheet
- Code comments in both languages where appropriate
- Wait time increased to 5+ seconds after questioning

PSEUDOCODE CONVENTION (Cambridge IGCSE Standard)

This curriculum uses Cambridge-standard pseudocode:

SET variable TO value

INPUT variable

OUTPUT "text"

OUTPUT variable

IF condition THEN

...

ELSE

```
...
ENDIF
FOR variable FROM start TO end
...
ENDFOR
WHILE condition DO
...
ENDWHILE
REPEAT
...
UNTIL condition
```

Note: Some resources may use <- (arrow) assignment; Cambridge uses SET/TO.

PYTHON ENVIRONMENT

IDE: Thonny (same as Y9S1 for consistency)

Python Version: 3.10+

All programs saved with descriptive filenames in student folders.

WEEK 1: ALGORITHMS & PROBLEM SOLVING INTRO

Overview: This week introduces the fundamental concept of algorithms — the heart of computer science. Students learn that an algorithm is a step-by-step solution to a problem, practise decomposing problems, and write their first pseudocode using Cambridge-standard notation. The week establishes the Input-Process-Output (IPO) model that underpins all programming.

Syllabus Ref: 2.1 Algorithm Design (0478/22)

LESSON 1.1 — WHAT IS AN ALGORITHM?

Topic: Problem Decomposition, IPO Model, First Pseudocode

Duration: 40 minutes

Lesson Type: Conceptual Introduction with Practical Writing

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to define an algorithm, decompose a simple problem into input, process, and output steps, and write basic pseudocode using Cambridge-standard notation.

Key Vocabulary:

- algorithm = 算法 (suàn fǎ)

- pseudocode = 伪代码 (wěi dài mǎ)

- decomposition = 分解 (fēn jiě)
- input = 输入 (shū rù)
- process = 处理 (chǔ lǐ)
- output = 输出 (shū chū)
- sequence = 顺序 (shùn xù)
- variable = 变量 (biàn liàng)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | Display on board: "Write down the steps to make a cup of instant noodles. Be VERY specific — every tiny step."
- | Students write in notebooks. Teacher circulates and picks
- | 2 students to share. Highlight: the need for precision and
- | correct ORDER. (2 min writing + 1 min sharing)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Definition: An algorithm is a step-by-step set of instructions to solve a problem.
- | Everyday algorithms: making tea, tying shoelaces, morning routine.
- | Key point: Computers need EVERY step. They cannot "figure it out."
- | (Slide 2) Problem Decomposition — breaking a big problem into smaller, manageable parts.
- | Example: "Calculate average test score"
- | → Break into: (1) Get all scores, (2) Add them up,
- | (3) Count how many scores, (4) Divide total by count
- | (Slide 3) The IPO Model:
- | 
- | Input = What the computer needs to know (data entered)
- | Process = What the computer does with the data (calculations)
- | Output = What the computer shows the user (result)
- | (Slide 4) Introduction to Pseudocode:
- | - Not a real programming language
- | - A way to plan your program before coding
- | - Uses plain English with structured keywords
- | Cambridge pseudocode keywords introduced today:
- | SET — creates a variable and gives it a value

- | INPUT — gets data from the user
- | OUTPUT — displays something on screen
- | Example on board:
- | SET Name TO "Li Wei"
- | OUTPUT "Hello, " + Name

12-25 min | GUIDED PRACTICE

- | Whole-class worked example:
- | "Write pseudocode to ask the user for two numbers, add them, and display the result."
- | Teacher models decomposition first:
- | INPUT: Two numbers from the user
- | PROCESS: Add the two numbers
- | OUTPUT: Show the sum
- | Then writes pseudocode line by line with student input:
- | SET Num1 TO 0
- | SET Num2 TO 0
- | SET Total TO 0
- | OUTPUT "Enter first number:"
- | INPUT Num1
- | OUTPUT "Enter second number:"
- | INPUT Num2
- | SET Total TO Num1 + Num2
- | OUTPUT "The total is: " + Total
- | Second example (pair work):
- | "Write pseudocode to ask for a person's name and age, then display a greeting message."
- | Pairs discuss for 2 min, then one pair shares.

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 1A — Section A: IPO identification
- | Students complete 5 IPO analysis questions identifying input, process, and output for given scenarios.
- | Then attempt to write pseudocode for 2 simple problems.
- | (Worksheet 1A distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "3-2-1 Exit Ticket"
- | 3 keywords you learned today

- | 2 things you understand well
- | 1 question you still have
- | Homework: Write pseudocode for: "Ask the user for the length and width of a rectangle, calculate the area, and display it."
- | Include IPO decomposition at the top.

EAL Support:

- Pre-printed bilingual word bank on desk (algorithm/pseudocode/IPO)
- Colour-coded IPO boxes (blue=input, green=process, orange=output)
- Sentence frame: "The _____ is the input because the user enters it."
- Partner EAL students with bilingual peers for pair discussion
- Visual diagram of IPO on every slide

Differentiation:

Emerging | Provide partially completed pseudocode with blanks to fill in.

- | Give a word bank of pseudocode keywords.
- | Use the instant noodles example for IPO identification.

Developing | Complete all worksheet tasks independently.

- | Write pseudocode from English descriptions.

Secure | Extension: Add error checking to pseudocode (e.g., check if number is negative). Write pseudocode for a 3-number average.

Resources Needed:

- Projector with slides (IPO diagram, pseudocode examples)
- Printed IPO diagram handouts (A5, colour)
- Worksheet 1A (printed, one per student)
- Whiteboard markers for student sharing
- Exit ticket slips

LESSON 1.2 — PSEUDOCODE PRACTICE: SEQUENCE ONLY

Topic: Sequence Constructs, Translating English to Pseudocode

Duration: 40 minutes

Lesson Type: Skill-Building Practice

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to write pseudocode using sequence constructs (SET, INPUT, OUTPUT) and translate English instructions into correct Cambridge-standard pseudocode.

Key Vocabulary:

- sequence = 顺序结构 (shùn xù jié gòu)
- construct = 结构/构造 (jié gòu / gòu zào)
- assign = 赋值 (fù zhí)
- concatenate = 连接 (lián jiē)
- literal = 字面量 (zì miàn liàng)
- string = 字符串 (zì fú chuàn)
- integer = 整数 (zhěng shù)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Spot the Error" — Display this pseudocode on board:
- | SET age TO 15
- | OUTPUT "Enter your age"
- | INPUT age
- | OUTPUT "Next year you will be" + age + 1
- | Ask: "There are TWO problems. Find them."
- | (Answers: OUTPUT should come after INPUT for the prompt;
- | "age + 1" needs brackets or the +1 will concatenate as string)
- | This activates prior learning and sets up today's focus.

3-12 min | DIRECT INSTRUCTION

- | Recap: Last lesson we learned SET, INPUT, and OUTPUT.
- | Today we practise using them correctly in SEQUENCE.
- | (Slide 1) The SEQUENCE structure:
- | - Statements execute ONE AFTER ANOTHER, top to bottom
- | - Order MATTERS enormously
- | - Example of wrong order:
- | OUTPUT "Your name is " + Name
- | INPUT Name
- | → This would fail because Name has no value yet!
- | (Slide 2) Rules of pseudocode:
- | 1. One statement per line
- | 2. Use meaningful variable names (not x, y, z)
- | 3. ALL keywords in UPPERCASE (Cambridge standard)
- | 4. Indent inside control structures (introduced next week)
- | (Slide 3) SET — assignment in detail:
- | SET Counter TO 0 ← gives Counter the value 0
- | SET Name TO "Mei Ling" ← stores text

- | SET Price TO 12.50 ← stores a decimal
- | (Slide 4) INPUT and OUTPUT combinations:
- | OUTPUT "Enter your name" ← prompt (tells user what to do)
- | INPUT UserName ← stores their answer
- | OUTPUT "Hello, " + UserName ← greeting with their name
- | (Slide 5) String concatenation:
- | Using + to join text:
- | OUTPUT "Total: " + Total ← joins "Total: " with the value
- | Real-world example:
- | "A shop wants a program to calculate the total cost of items."
- | SET ItemPrice TO 0
- | SET Quantity TO 0
- | SET TotalCost TO 0
- | OUTPUT "Enter item price:"
- | INPUT ItemPrice
- | OUTPUT "Enter quantity:"
- | INPUT Quantity
- | SET TotalCost TO ItemPrice * Quantity
- | OUTPUT "Total cost: " + TotalCost

12-25 min | GUIDED PRACTICE

- | "Translate to Pseudocode" — Teacher reads English, students write pseudocode, then check together.
- | Exercise 1: "Ask the user for their height in cm and their weight in kg. Calculate BMI (weight / height / height * 10000) and display it."
- | (Work through together — 4 min)
- | Exercise 2: "A student has scores of 85, 90, and 78. Calculate the average and display it with the message 'Your average is:'"
- | (Students attempt in pairs, then one pair shares — 4 min)
- | Exercise 3: "Ask for the price of a meal and the tip percentage. Calculate the tip amount and the total bill. Display both."
- | (Students attempt individually, check with partner — 5 min)

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 1B — Pseudocode writing practice
- | Section A: 5 "fill in the blank" pseudocode exercises
- | Section B: 3 "write from description" exercises

| Section C: 1 exam-style question (4 marks)

| (Worksheet 1B distributed — see below)

35-40 min | PLENARY + HOMEWORK

| Plenary: Thumbs up/middle/down — "How confident are you writing

| pseudocode from an English description?"

| Quick-fire: Teacher says English, students say first line of

| pseudocode as a chorus.

| Homework: Write pseudocode for: a program that asks for the

| radius of a circle and calculates the circumference

| (Circumference = $2 * 3.14 * \text{radius}$). Use the IPO model.

EAL Support:

- Sentence frames for pseudocode writing:

"The input is _____, so I use INPUT _____."

"I need to store _____, so I SET _____ TO _____."

- Highlight that "SET...TO" means "give the value to" (把...设为)

- Colour-code keywords: SET (green), INPUT (blue), OUTPUT (orange)

- Word bank available on worksheet

Differentiation:

Emerging | Sentence starters for every line of pseudocode.

| Matching activity: match English description to pseudocode line.

| Only complete Section A of worksheet.

Developing | Complete all sections of worksheet independently.

| Write pseudocode from English descriptions.

Secure | Extension: Write pseudocode for a currency converter that

| converts RMB to USD (rate: 1 USD = 7.2 RMB). Include formatted

| output to 2 decimal places. Then add a second conversion (RMB

| to HKD at rate 1 HKD = 0.92 RMB).

Resources Needed:

- Slide deck with pseudocode examples

- Worksheet 1B (one per student)

- Whiteboard for modelling

- Mark scheme printed for teacher reference

WORKSHEET 1A — INTRODUCTION TO ALGORITHMS & IPO

Name: _____ Class: _____ Date: _____

Learning Objective: I can identify the input, process, and output of a

problem and understand what an algorithm is.

Bilingual Support:

- algorithm = 算法 (suàn fǎ)
- input = 输入 (shū rù)
- process = 处理 (chǔ lǐ)
- output = 输出 (shū chū)
- variable = 变量 (biàn liàng)
- pseudocode = 伪代码 (wěi dài mǎ)

SECTION A: What is an Algorithm? [6 marks]

1. Circle the best definition of an algorithm: [1 mark]

- a) A computer program written in Python
- b) A step-by-step set of instructions to solve a problem
- c) A type of computer virus
- d) A diagram showing computer hardware

1. Give TWO examples of algorithms you use in everyday life:

(do not use examples from the lesson)

Example 1: _____ [1 mark]

Example 2: _____ [1 mark]

2. Explain why the ORDER of steps in an algorithm is important. [2 marks]

(Use the instant noodles or another example to help explain.)

4. What is problem decomposition? [1 mark]

SECTION B: Input-Process-Output (IPO) Analysis [8 marks]

For each problem, identify the INPUT, PROCESS, and OUTPUT.

5. A program calculates the total cost of buying 3 books.

INPUT: _____ [1]

PROCESS: _____ [1]

OUTPUT: _____ [1]

6. A program converts Celsius temperature to Fahrenheit.

(Formula: Fahrenheit = Celsius $\times$ 9/5 + 32)

INPUT: _____ [1]

PROCESS: _____ [1]

OUTPUT: _____ [1]

3. A program asks for your test score (out of 100) and tells you if you passed (score ≥ 60).

INPUT: _____ [1]

PROCESS: _____ [1]

OUTPUT: _____ [1]

SECTION C: Writing Pseudocode [6 marks]

4. Write pseudocode to ask the user for their name and their favourite colour, then display a message: "Hello [name], your favourite colour is [colour]."

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

WORKSHEET 1B — PSEUDOCODE: SEQUENCE PRACTICE

Name: _____ Class: _____ Date: _____

Learning Objective: I can write pseudocode using SET, INPUT, and OUTPUT in the correct sequence.

Bilingual Support:

- SET = 赋值 (fù zhí)
- INPUT = 输入 (shū rù)
- OUTPUT = 输出 (shū chū)
- sequence = 顺序 (shùn xù)
- variable = 变量 (biàn liàng)
- concatenate = 连接 (lián jiē)

SECTION A: Fill in the Blanks [5 marks]

Complete the pseudocode by filling in the missing parts.

1. A program asks for a user's age and displays it.

SET _____ TO 0

OUTPUT "Please enter your age:"

INPUT _____

OUTPUT "You are " + _____ + " years old."

2. A program calculates the area of a rectangle.

```
SET Length TO 0
SET Width TO 0
SET Area TO 0
OUTPUT "Enter length:"
INPUT _____
OUTPUT "Enter width:"
INPUT _____
SET Area TO _____ * _____
OUTPUT "The area is " + _____
```

3. A program calculates the cost of petrol for a trip.

```
SET Distance TO 0
SET FuelEfficiency TO 0
SET FuelPrice TO 0
SET TotalCost TO 0
OUTPUT "Enter trip distance in km:"
INPUT Distance
OUTPUT "Enter fuel efficiency (km per litre):"
INPUT FuelEfficiency
OUTPUT "The total fuel cost is: " + TotalCost
```

SECTION B: Write Pseudocode from Description [9 marks]

Write complete pseudocode for each problem.

5. A program asks for the user's first name and last name, then displays their full name.

```
_____ [1]
_____ [1]
_____ [1]
_____ [1]
_____ [1]
```

6. A shop sells pens for 5 RMB each. The program asks how many pens the customer wants, then calculates and displays the total cost.

```
_____ [1]
_____ [1]
_____ [1]
```

_____ [1]

7. A program asks for a temperature in Celsius and converts it to Kelvin ($\text{Kelvin} = \text{Celsius} + 273$). Display the result.

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

SECTION C: Exam-Style Question [4 marks]

8. A teacher wants a program to calculate the average mark for three tests. Write pseudocode for this program.

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

ANSWER KEY — WEEK 1

WORKSHEET 1A:

SECTION A:

9. b) A step-by-step set of instructions to solve a problem [1]

10. Any two valid examples e.g., following a recipe, brushing teeth, getting to school, assembling furniture [1 each]

11. Mark for explaining that steps must be in correct order [1] and giving a relevant example [1]. E.g., "If you add hot water before the noodles are in the bowl, the water spills and the noodles are not cooked. Order matters because each step depends on the previous one being complete."

12. Breaking a large problem into smaller, more manageable parts [1]

SECTION B:

5. INPUT: Price of each book (or: three book prices) [1]

PROCESS: Add the three prices together [1]

OUTPUT: The total cost [1]

6. INPUT: Temperature in Celsius [1]

PROCESS: Multiply by 9/5, then add 32 [1]

OUTPUT: Temperature in Fahrenheit [1]

7. INPUT: Test score [1]

PROCESS: Check if score is greater than or equal to 60 [1]

OUTPUT: Pass or Fail message [1]

SECTION C:

8. Accept any logically correct pseudocode. Example answer:

SET FirstName TO "" [1]

SET FavColour TO "" [1]

OUTPUT "Enter your name:" [1]

INPUT FirstName [1]

OUTPUT "Enter your favourite colour:" [1]

INPUT FavColour [1]

OUTPUT "Hello " + FirstName + ", your favourite colour is " + FavColour [1]

WORKSHEET 1B:

SECTION A:

1. Age / Age / Age (accept UserAge) [1 for all correct]

2. Length / Width / Length * Width / Area [1]

13. SET TotalCost TO (Distance / FuelEfficiency) * FuelPrice [1]

(Accept any correct formula for fuel cost calculation)

SECTION B:

4. Example answer:

SET FirstName TO "" [1]

SET LastName TO "" [1]

OUTPUT "Enter first name:" [1]

INPUT FirstName [1]

OUTPUT "Enter last name:" [1]

INPUT LastName [1]

OUTPUT "Your full name is: " + FirstName + " " + LastName [1]

5. Example answer:

SET PenPrice TO 5 [1]

SET Quantity TO 0

SET Total TO 0

OUTPUT "How many pens?" [1]

INPUT Quantity [1]

SET Total TO PenPrice * Quantity [1]

OUTPUT "Total cost: " + Total [1]

6. Example answer:

SET Celsius TO 0 [1]

SET Kelvin TO 0

OUTPUT "Enter temperature in Celsius:" [1]

INPUT Celsius [1]

SET Kelvin TO Celsius + 273 [1]

OUTPUT "Temperature in Kelvin: " + Kelvin [1]

(Give full marks for any logically correct approach)

SECTION C:

7. Example answer:

SET Mark1 TO 0 [1]

SET Mark2 TO 0

SET Mark3 TO 0

SET Average TO 0

OUTPUT "Enter mark for test 1:" [1]

INPUT Mark1 [1]

OUTPUT "Enter mark for test 2:" [1]

INPUT Mark2 [1]

OUTPUT "Enter mark for test 3:" [1]

INPUT Mark3 [1]

SET Average TO (Mark1 + Mark2 + Mark3) / 3 [1]

OUTPUT "The average mark is: " + Average [1]

Note: Accept with or without brackets. Accept sum stored in intermediate variable. Deduct 1 mark if /3 is missing.

WEEK 2: FLOWCHARTS

Overview: This week introduces flowcharts as a visual alternative to pseudocode for representing algorithms. Students learn standard flowchart symbols, practise creating flowcharts for sequence programs, and then progress to flowcharts with selection (decision/diamond). By the end of the week, students can convert between pseudocode and flowcharts and understand when each representation is most appropriate.

Syllabus Ref: CIE 0478 Topic 2.1 (Algorithm Design — Flowcharts)

LESSON 2.1 — FLOWCHART SYMBOLS & SEQUENCE

Topic: Flowchart Symbols, Creating Sequence Flowcharts

Duration: 40 minutes

Lesson Type: Visual/Practical with Hands-on Drawing

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to identify and use standard flowchart symbols (start/end, process, input/output, decision) and create a flowchart for a simple sequence algorithm.

Key Vocabulary:

- flowchart = 流程图 (liú chéng tú)
- symbol = 符号 (fú hào)
- terminal = 起止框 (qǐ zhǐ kuāng)
- process box = 处理框 (chǔ lǐ kuāng)
- input/output box = 输入输出框 (shū rù shū chū kuāng)
- decision diamond = 判断菱形 (pàn duàn líng xíng)
- flow line = 流向线 (liú xiàng xiàn)
- arrow = 箭头 (jiàn tóu)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

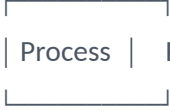
- | Display a simple flowchart on the board with 5 symbols.
- | Ask: "What do you think each shape means? Write your best guess for each shape." Students write in notebooks. This activates prior knowledge and creates curiosity about the symbols.

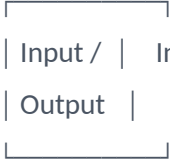
3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Introduction: "Pseudocode is like a script — text-based.
- | Flowcharts are like a map — visual. Both show the same algorithm."

| (Slide 2) Standard Flowchart Symbols (Cambridge IGCSE):

|  | Terminal (rounded rectangle) — start or end

|  | Process (rectangle) — calculation or action

|  | Input / | Input/Output (parallelogram) — user I/O
| Output |

|  | Decision (diamond) — yes/no question

| / \ (introduced in Lesson 2.2)

| / \ |

| → Arrow lines show the flow of control

| Cambridge ONLY uses these four symbol types.

| Other symbols exist in industry, but we use these for the exam.

| (Slide 3) Rules for drawing flowcharts:

| 1. Every flowchart starts with START and ends with END

| 2. Flow lines must have ARROWS to show direction

| 3. Only ONE flow line comes out of each symbol (except
| decision, which has two: YES and NO)

| 4. Keep it neat — use a ruler or digital tool

| (Slide 4) Worked example — flowchart for adding two numbers:

| [START]

| [SET Num1 TO 0]

| [SET Num2 TO 0]

| [SET Total TO 0]

| [/ Enter first \]

| [\ number: /]

| [\ Num1 /]

| [/ Enter second \]

| [\ number: /]

| [\ Num2 /]

| [Total = Num1 + Num2]

| [/ Output Total \]

| [END]

12-25 min | GUIDED PRACTICE

| "Draw the flowchart" — students use paper templates or draw.io.

| Exercise 1 (teacher models on board):

| "Create a flowchart that asks for a user's name and age, then

| displays 'Hello [name], you are [age] years old.'"

| Steps modelled:

| START → SET Name="", SET Age=0 → OUTPUT "Enter name"

| → INPUT Name → OUTPUT "Enter age" → INPUT Age

| → OUTPUT "Hello "+Name+", you are "+Age+" years old"

| → END

| Exercise 2 (pairs):

| "Draw a flowchart for: Ask for length and width of a rectangle,

| calculate area, display area."

| Pairs work on A4 paper with template. Teacher circulates and

| gives feedback on symbol usage and arrow direction.

25-35 min | INDEPENDENT PRACTICE

| Worksheet 2A — Section A: Match symbols to names

| Section B: Identify errors in given flowcharts

| Section C: Draw a flowchart from pseudocode

| (Worksheet 2A distributed)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Symbol Sprint" — Teacher calls out a description,

| students draw the correct symbol in the air with their finger.

| (e.g., "Start of program!" → draw rounded rectangle)

| Homework: Using draw.io (or paper), draw a flowchart for:

| "Ask for the price of an item and the quantity. Calculate total

| cost and display it." Label each symbol with its name.

EAL Support:

- Laminated flowchart symbol reference card on every desk
- Colour-coded symbols: start/end (red), process (blue), I/O (green), decision (yellow)
- Students label each symbol on their flowchart in BOTH English and Chinese
- Sentence frame: "This is a _____ symbol. It means _____."
- Visual matching game at start of guided practice

Differentiation:

Emerging | Pre-drawn flowchart shapes to cut out and arrange in order.

| Only sequence flowcharts (no decisions).

| Complete Section A and B only on worksheet.

Developing | Draw flowcharts independently using paper or draw.io.

| Complete all worksheet sections.

Secure | Extension: Draw a flowchart for a multi-step calculation

| (e.g., BMI calculator with 3 inputs). Compare with pseudocode

| version — which is clearer? Write 3 sentences explaining

| when flowcharts are better than pseudocode.

Resources Needed:

- Slide deck with flowchart symbols
- Printed flowchart template paper (A4 with light grid)
- Laminated symbol reference cards
- Scissors and glue for Emerging tier
- draw.io access (or alternative: lucidchart, or paper)
- Coloured pencils/markers

LESSON 2.2 — FLOWCHARTS WITH SELECTION (DECISION)

Topic: Decision Diamonds, IF...THEN...ELSE in Flowcharts, Comparing

Flowcharts vs Pseudocode

Duration: 40 minutes

Lesson Type: Conceptual Extension with Comparison

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to create flowcharts that include selection using the decision diamond, convert between pseudocode with IF...THEN...ELSE and flowcharts, and evaluate when to use each representation.

Key Vocabulary:

- selection = 选择结构 (xuǎn zé jié gòu)
- decision = 判断 (pàn duàn)
- branch = 分支 (fēn zhī)
- condition = 条件 (tiáo jiàn)
- IF...THEN...ELSE = 如果...那么...否则 (rú guǒ...nà me...fǒu zé)
- YES branch = 是分支 (shì fēn zhī)
- NO branch = 否分支 (fǒu fēn zhī)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| Display a flowchart with a decision diamond:

| [START] → [INPUT Age] → ◇Age ≥ 18?◇ → (YES)→[OUTPUT "Adult"]

| → (NO) →[OUTPUT "Child"]

| → [END]

| Ask: "What do you think the diamond does? What happens if the

| user enters 20? What if they enter 15?"

| Students discuss in pairs for 2 minutes.

3-12 min | DIRECT INSTRUCTION

| (Slide 1) The Decision Diamond:

- | - Diamond shape = a question with YES or NO answer
- | - Two paths come OUT of the diamond
- | - Each path must eventually rejoin (or go to END)

| ◇ Condition? ◇

| / YES NO \ | / \ | [Do this] [Do that]

| \ /

| \ /

| [Continue]

| (Slide 2) Mapping IF...THEN...ELSE to flowchart:

Pseudocode:	Flowchart:
IF Age >= 18 THEN	[INPUT Age]
OUTPUT "Adult"	↓
ELSE	◇Age>=18?◇
OUTPUT "Minor"	/YES \NO
ENDIF	[Adult] [Minor]
	\ /
	\ /
	[END]

| (Slide 3) Rules for decision diamonds:

- 1. The condition must be a YES/NO (true/false) question
- 2. Label each branch YES/NO (or TRUE/FALSE)
- 3. Both branches must continue somewhere — cannot stop
- 4. Use meaningful conditions (Age >= 18, not "x > y")

| (Slide 4) Comparing Flowcharts vs Pseudocode:

| Flowcharts are BETTER when:

- | - Visualising the flow of control
- | - Showing decisions and loops clearly
- | - Presenting to non-programmers
- | Pseudocode is BETTER when:
 - | - Writing detailed logic quickly
 - | - The algorithm is long or complex
 - | - Preparing to write actual code
- | Cambridge exams accept BOTH. Students should be comfortable
- | with both representations.

12-25 min | GUIDED PRACTICE

- | Exercise 1 (teacher models):
 - | "Convert this pseudocode to a flowchart:"
 - | SET Score TO 0
 - | OUTPUT "Enter your test score:"
 - | INPUT Score
 - | IF Score >= 60 THEN
 - | OUTPUT "You passed!"
 - | ELSE
 - | OUTPUT "You need to retake."
 - | ENDIF
- | Draw flowchart step by step on board with student input.
- | Exercise 2 (pairs):
 - | "Convert this flowchart to pseudocode" (display on slide)
 - | Flowchart: START → INPUT Temperature → ◇Temp > 30?◇
 - | YES → OUTPUT "Hot" → END
 - | NO → OUTPUT "Not hot" → END
- | Exercise 3 (individual attempt):
 - | "Draw a flowchart for: Ask the user for a password. If the
 - | password equals 'Shenzhen2024', output 'Access granted'.
 - | Otherwise, output 'Access denied'."

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 2B — Section A: Convert pseudocode to flowchart
- | Section B: Convert flowchart to pseudocode
- | Section C: Comparison question + exam-style question
- | (Worksheet 2B distributed)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Would You Rather?"
- | Display a complex algorithm. Ask: "Would you rather represent this in pseudocode or a flowchart? Why?"
- | Students vote with cards (P or F) and justify.
- | Homework: Create BOTH a pseudocode AND a flowchart for: "A program asks the user for their exam mark (0-100). If the mark is 90 or above, output 'A'. Otherwise, output 'Not A'."
- | Write one sentence comparing which was easier to create.

EAL Support:

- YES/NO labels on decision diamond in BOTH languages:
YES = 是 (shì), NO = 否 (fǒu)
- Sentence frames for comparison:
"I prefer _____ because it is easier to _____."
"Flowcharts show the _____ clearly."
"Pseudocode is better for _____."
- Colour-code the two branches differently (YES = green, NO = red)
- Pre-printed sentence starters for the comparison question

Differentiation:

- Emerging | Use pre-cut flowchart shapes including a diamond.
 - | Only complete one conversion (pseudocode → flowchart).
 - | Matching activity for symbol identification.
- Developing | Complete both conversions independently.
 - | Write the comparison paragraph.
- Secure | Extension: Create a flowchart with TWO decision diamonds
 - | (nested selection). E.g., exam grade calculator with boundaries at 90, 70, 50. Convert to pseudocode and explain which representation was clearer for this complex case and why.

Resources Needed:

- Slide deck with flowchart examples
- Pre-cut flowchart shapes (Emerging tier)
- Worksheet 2B (one per student)
- Voting cards (P/F) for plenary
- draw.io access for digital drawing option

Name: _____ Class: _____ Date: _____

Learning Objective: I can identify flowchart symbols and draw sequence flowcharts correctly.

Bilingual Support:

- flowchart = 流程图 (liú chéng tú)
- terminal = 起止框 (qǐ zhǐ kuāng)
- process = 处理框 (chǔ lǐ kuāng)
- input/output = 输入输出框 (shū rù shū chū kuāng)
- decision = 判断菱形 (pàn duàn líng xíng)
- arrow = 箭头 (jiàn tóu)

SECTION A: Match the Symbols [5 marks]

Draw a line to match each symbol name to its correct shape.

- | | |
|--------------------------------|---|
| 1. Terminal (Start/End) | a.  (rectangle) |
| 2. Process | b.  (parallelogram) |
| 3. Input/Output | c.  (rounded rectangle) |
| 4. Decision | d.  (diamond) |
| 5. Flow line | e.  (arrow) |

SECTION B: Identify the Errors [6 marks]

The flowchart below contains THREE errors. Find and describe them.

[START]



[OUTPUT "Enter name"]



[INPUT Name]



[OUTPUT "Hello " + Name]



[END]

Error 1: _____ [2]

Error 2: _____ [2]

Error 3: _____ [2]

SECTION C: Draw a Flowchart [4 marks]

6. Draw a flowchart for this pseudocode:

```
SET Num1 TO 0
SET Num2 TO 0
SET Num3 TO 0
SET Average TO 0
OUTPUT "Enter three numbers:"
INPUT Num1
INPUT Num2
INPUT Num3
SET Average TO (Num1 + Num2 + Num3) / 3
OUTPUT "The average is: " + Average
```

Use the space below. Remember to use the correct symbols and arrows.

WORKSHEET 2B — FLOWCHARTS WITH SELECTION

Name: _____ Class: _____ Date: _____

Learning Objective: I can create flowcharts with decisions and convert between pseudocode and flowcharts.

Bilingual Support:

- selection = 选择 (xuǎn zé)
- condition = 条件 (tiáo jiàn)
- IF...THEN...ELSE = 如果...那么...否则
- branch = 分支 (fēn zhī)
- YES/NO = 是/否 (shì/fǒu)

SECTION A: Pseudocode to Flowchart [6 marks]

1. Draw a flowchart for this pseudocode:

```
SET Temperature TO 0
OUTPUT "Enter the temperature:"
INPUT Temperature
IF Temperature > 25 THEN
    OUTPUT "It is warm"
ELSE
    OUTPUT "It is not warm"
ENDIF
```


SECTION B: Flowchart to Pseudocode [6 marks]

2. Write the pseudocode for this flowchart:

```
[START]
↓
[/ Enter score \]
↓
[\ out of 100 /]
↓
◇ Score >= 50? ◇
YES ↙      ↘ NO
[/ You passed! \] [/ You failed. \]
[END]
```

SECTION C: Comparison & Exam Question [8 marks]

14. When is a flowchart better than pseudocode for showing an algorithm?

Give TWO reasons. [2 marks]

15. When is pseudocode better than a flowchart? Give TWO reasons. [2 marks]

5. A student writes this pseudocode for a login check:

```
SET Password TO ""
OUTPUT "Enter password:"
INPUT Password
IF Password == "Shenzhen2024" THEN
    OUTPUT "Welcome!"
```

ELSE

 OUTPUT "Wrong password"

ENDIF

Draw the flowchart for this algorithm in the space below. [4 marks]

ANSWER KEY — WEEK 2

WORKSHEET 2A:

SECTION A:

16. c (rounded rectangle) [1]

17. a (rectangle) [1]

18. b (parallelogram) [1]

19. d (diamond) [1]

20. e (arrow) [1]

SECTION B:

Accept any three valid errors. Common answers:

- Error 1: The flow line between the last output and END has no arrow [2]

(or: arrow is missing from the vertical line)

- Error 2: Variable Name is not initialised with SET before INPUT [2]

(Cambridge standard: variables should be declared/initialised before use)

- Error 3: No prompt message before INPUT [2]

(Best practice: user should know what to enter)

Alternative acceptable errors:

- The START/END boxes are not rounded rectangles [2]

- The INPUT should be in a parallelogram, not rectangle [2]

- Missing variable declaration/initialisation at start [2]

SECTION C:

6. Marking criteria:

- Correct START and END symbols (rounded rectangles) [1]
- Correct use of process boxes for SET statements [1]
- Correct use of input/output boxes for OUTPUT and INPUT [1]
- Correct flow with arrows in right direction [1]
- Deduct 1 mark for each missing or incorrect symbol (max 4)

WORKSHEET 2B:

SECTION A:

1. Marking criteria for the flowchart:

- START and END in rounded rectangles [1]

- SET Temperature TO 0 in process box [1]
- OUTPUT and INPUT in parallelograms [1]
- Decision diamond with correct condition (Temperature > 25) [1]
- YES branch leads to "It is warm" output [1]
- NO branch leads to "It is not warm" output [1]

(Total: max 6 marks for correct complete flowchart)

SECTION B:

2. SET Score TO 0 [1]

OUTPUT "Enter score out of 100" [1]

INPUT Score [1]

IF Score >= 50 THEN [1]

 OUTPUT "You passed!" [1]

ELSE [1]

 OUTPUT "You failed." [1]

ENDIF [1]

SECTION C:

3. Any two valid reasons, e.g.:

- Flowcharts are visual and easier to understand at a glance [1]
- Flowcharts show the flow of control clearly with arrows [1]
- Good for presenting to people who cannot read pseudocode [1]
- Decisions and loops are easier to visualise [1]

4. Any two valid reasons, e.g.:

- Pseudocode is quicker to write [1]
- Pseudocode can show more detail in less space [1]
- Pseudocode is closer to real programming code [1]
- Better for complex algorithms that would need a very large flowchart [1]

5. Marking criteria for flowchart:

- START and END present and correct [1]
- SET Password TO "" in process box [1]
- OUTPUT and INPUT in parallelograms [1]
- Decision diamond with condition Password == "Shenzhen2024" [1]
- Both YES and NO branches lead to appropriate outputs [1]
- Arrows show correct flow direction throughout [1]

(Max 4 marks — give marks for best fit)

WEEK 3: SEARCHING ALGORITHMS

Overview: This week introduces two fundamental searching algorithms — linear search and binary search. Students learn how each algorithm works through physical card-sorting activities and trace tables, write pseudocode for both, and compare their efficiency. This is a key topic for Paper 2 and frequently appears in exam questions. Students must understand when each algorithm is appropriate and be able to trace through both manually.

Syllabus Ref: CIE 0478 Topic 2.1 (Algorithm Design — Searching)

LESSON 3.1 — LINEAR SEARCH

Topic: Linear Search Algorithm, Trace Tables, Complexity

Duration: 40 minutes

Lesson Type: Algorithmic Thinking with Physical Demonstration

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to explain how linear search works, trace through a linear search using a trace table, write linear search pseudocode, and identify best, average, and worst case scenarios.

Key Vocabulary:

- linear search = 线性搜索 (xiàn xìng sōu suǒ)
- search algorithm = 搜索算法 (sōu suǒ suàn fǎ)
- element = 元素 (yuán sù)
- array/list = 数组/列表 (shù zǔ / liè biǎo)
- index = 索引 (suǒ yǐn)
- target = 目标 (mù biāo)
- found = 找到 (zhǎo dào)
- not found = 未找到 (wèi zhǎo dào)
- iteration = 迭代 (dié dài)
- trace table = 追踪表 (zhuī zōng biǎo)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You have a shuffled deck of cards (Ace through King of hearts).
- | You need to find the 7 of hearts. Describe your method."
- | Students write 2-3 sentences. This introduces the concept of
- | systematic searching and sets up the linear search idea.

3-12 min | DIRECT INSTRUCTION

| (Slide 1) Searching Algorithms:
| "Searching is one of the most common tasks in computing.
| Every time you search Google, look up a contact, or scan
| a QR code — a searching algorithm is working."

| (Slide 2) Linear Search:
| "The simplest search. Start at the beginning. Check each
| item one by one until you find what you are looking for."
| Visual demonstration with physical cards on desk:
| Cards: [3] [7] [1] [9] [4] — searching for 9
| Step 1: Check card 1 → 3? No. Move to next.
| Step 2: Check card 2 → 7? No. Move to next.
| Step 3: Check card 3 → 1? No. Move to next.
| Step 4: Check card 4 → 9? YES! Found at position 4.

| (Slide 3) Linear Search Pseudocode:
| SET Found TO FALSE
| SET Index TO 0
| SET Target TO 9
| SET List TO [3, 7, 1, 9, 4]
| WHILE Found == FALSE AND Index < 5 DO
| IF List[Index] == Target THEN
| SET Found TO TRUE
| OUTPUT "Found at position " + Index
| ELSE
| SET Index TO Index + 1
| ENDIF
| ENDWHILE
| IF Found == FALSE THEN
| OUTPUT "Not found"
| ENDIF

| (Slide 4) Trace Tables:
| "A trace table shows the value of each variable at each step.
| It helps us understand what the algorithm is doing."
| Example trace table for searching 9 in [3, 7, 1, 9, 4]:
	Step	Index	List[Index]	Target	Found	Action
	Init	0	-	9	FALSE	Start
	1	0	3	9	FALSE	Increment

2	1	7	9	FALSE	Increment
3	2	1	9	FALSE	Increment
4	3	9	9	TRUE	Found!

(Slide 5) Complexity:

- Best case: Target is first item → 1 comparison
- Average case: Target is in the middle → $n/2$ comparisons
- Worst case: Target is last or not present → n comparisons
- (n = number of items in the list)

12-25 min | GUIDED PRACTICE

- Activity: "Human Linear Search"
 - 5 students stand at front holding numbered cards: [12] [5] [8] [3] [15]
 - Class searches for 8 using linear search. One student is the "pointer" (Index), moving along the line.
 - Together, fill in trace table on board:
- | Step | Index | Value | Target (8) | Found? |
|------|-------|-------|------------|--------|
| Init | 0 | - | 8 | FALSE |
| 1 | 0 | 12 | 8 | FALSE |
| 2 | 1 | 5 | 8 | FALSE |
| 3 | 2 | 8 | 8 | TRUE |
- Second round: Search for 20 (not in list)
 - Fill trace table — emphasise that ALL items must be checked when target is not found (worst case).

25-35 min | INDEPENDENT PRACTICE

- Worksheet 3A — Section A: Trace a linear search on paper
- Section B: Write linear search pseudocode
- Section C: Exam-style question with trace table
- (Worksheet 3A distributed)

35-40 min | PLENARY + HOMEWORK

- Plenary: "Linear Search Summary" — students complete on mini WB:
 - Linear search starts at the _____ of the list.
 - It checks each item _____ by _____.
 - Best case = _____ comparisons.
 - Worst case = _____ comparisons.
- Homework: Create a trace table for linear search of target = 50 in this list: [10, 30, 50, 70, 90, 110]
- Also state: best case, worst case, and how many comparisons

| were actually needed.

EAL Support:

- Physical card demonstration makes the abstract concrete
- Trace table template pre-printed with column headers
- Sentence frame: "At step ____, Index is ____, so we compare ____ to ____."
- Colour-code: Index column (blue), Found column (green/red)
- Word bank on worksheet with all key terms in both languages
- Pair EAL students with confident peers for the human search activity

Differentiation:

Emerging | Pre-filled trace table — students only fill in 2-3 rows.

- | Shorter list to search (3 items only).
- | Sentence starter for pseudocode writing.

Developing | Complete full trace table independently.

- | Write pseudocode from scratch.
- | Answer comparison questions.

Secure | Extension: Write a modified linear search that counts how

- | many times the target appears (not just finds first).
- | Also state the time complexity in Big-O notation: $O(n)$.
- | Research: When would linear search be the ONLY option?

Resources Needed:

- Numbered cards (A5, large) for physical demonstration
- Slide deck with pseudocode and trace table examples
- Worksheet 3A (one per student)
- Mini whiteboards for plenary
- Pre-printed trace table templates

LESSON 3.2 — BINARY SEARCH

Topic: Binary Search Algorithm, Comparison with Linear Search, Pseudocode

Duration: 40 minutes

Lesson Type: Conceptual with Comparison and Practice

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to explain how binary search works on sorted data, trace through a binary search with a trace table, compare linear and binary search, and write pseudocode for both.

Key Vocabulary:

- binary search = 二分搜索 (èr fēn sōu suǒ)
- sorted = 已排序 (yǐ pái xù)
- middle element = 中间元素 (zhōng jiān yuán sù)
- divide and conquer = 分治法 (fēn zhì fǎ)
- midpoint = 中点 (zhōng diǎn)
- discard = 丢弃 (diū qì)
- half = 一半 (yī bàn)
- efficiency = 效率 (xiào lǜ)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You are looking up the word 'algorithm' in an English dictionary. The dictionary has 1000 pages. Do you start at page 1 and check every word? Explain your method."
- | Students write for 2 minutes. This introduces the idea of splitting the search space — the core of binary search.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Binary Search:
- | "For sorted data only! Instead of checking one by one, we repeatedly divide the list in half."
- | Rule: The list MUST be sorted first!
- | If not sorted → binary search will NOT work.
- | (Slide 2) How it works — step by step:
- | List: [3, 7, 12, 15, 21, 25, 30, 35] Target: 25
- | Step 1: Find middle of entire list → index 3 (value 15)
- | Is 15 == 25? No. Is 15 > 25? No.
- | So 25 must be in the RIGHT half.
- | Discard left half: [21, 25, 30, 35]
- | Step 2: Find middle of remaining list → index 5 (value 25)
- | Is 25 == 25? YES! Found at index 5.
- | Only 2 comparisons instead of 6 for linear search!
- | (Slide 3) Physical demonstration:
- | 8 students at front with sorted cards: [10] [20] [30] [40]
- | [50] [60] [70] [80]
- | Search for 60.
- | Pointer 1 (Low) at start, Pointer 2 (High) at end.
- | Calculate Mid = (Low + High) // 2 = (0+7)//2 = 3 → card 40


```

| 40 < 60, so move Low to position 4.
| Mid = (4+7)//2 = 5 → card 60. Found!
| (Slide 4) Binary Search Pseudocode:
| SET List TO [10, 20, 30, 40, 50, 60, 70, 80]
| SET Target TO 60
| SET Low TO 0
| SET High TO 7
| SET Found TO FALSE
| WHILE Found == FALSE AND Low <= High DO
|   SET Mid TO (Low + High) // 2
|   IF List[Mid] == Target THEN
|     SET Found TO TRUE
|     OUTPUT "Found at position " + Mid
|   ELSE IF List[Mid] < Target THEN
|     SET Low TO Mid + 1
|   ELSE
|     SET High TO Mid - 1
|   ENDIF
| ENDWHILE
| IF Found == FALSE THEN
|   OUTPUT "Not found"
| ENDIF

```

| (Slide 5) Linear vs Binary — comparison table:

Feature	Linear Search	Binary Search
Data required	Any order	Must be sorted
Best case	1 comparison	1 comparison
Worst case	n comparisons	$\log_2(n)$ comparisons
Efficiency	Slower for large lists	Much faster for large sorted lists

12-25 min | GUIDED PRACTICE

```

| Trace table exercise (whole class):
| List: [2, 5, 8, 12, 16, 23, 38] Target: 16
| Fill trace table together:
| | Step | Low | High | Mid | List[Mid] | Action |
| | Init | 0 | 6 | - | - | Start |
| | 1 | 0 | 6 | 3 | 12 | Low = Mid+1 |

```

| 2 | 4 | 6 | 5 | 23 | High = Mid-1 |
 | 3 | 4 | 4 | 4 | 16 | Found! |
 | "Only 3 comparisons! Linear search would need 5."
 | Second trace: Target = 10 (not in list)
 | Students work in pairs, then check.

25-35 min | **INDEPENDENT PRACTICE**

| Worksheet 3B — Section A: Binary search trace tables
 | Section B: Comparison questions
 | Section C: Pseudocode writing and exam question
 | (Worksheet 3B distributed)

35-40 min | **PLENARY + HOMEWORK**

| Plenary: "Two Truths and a Lie" about searching:
 | 1. Linear search works on unsorted data.
 | 2. Binary search always finds the target in 1 comparison.
 | 3. Binary search needs the data to be sorted.
 | (Answer: #2 is the lie — it takes up to $\log_2(n)$ comparisons)
 | Homework: A list has 128 items. How many comparisons in the
 | worst case for linear search? For binary search? Show working.
 | Also: list TWO real-world examples where binary search is used.

EAL Support:

- Physical demonstration with student "pointers" for Low, Mid, High
- Colour-coded trace table: Low (blue), Mid (yellow), High (red)
- Sentence frame: "The middle element is _____. It is _____ than the target, so we discard the _____ half."
- Pre-printed comparison table with blanks to fill in
- Visual diagram showing list being halved at each step
- Chinese translation of "divide and conquer" = 分治法 (fēn zhì fǎ)

Differentiation:

Emerging | Pre-filled trace table with only Mid column to complete.

| Shorter list (4 items only).
 | Match-up activity: match search type to description.

Developing | Complete full trace tables.

| Answer comparison questions.
 | Write pseudocode from description.

Secure | Extension: Calculate $\log_2(n)$ for lists of size 16, 256,
 | 1024, 1,000,000. Write pseudocode for a recursive binary

- | search (advanced — introduces recursion concept).
- | Research: What searching algorithm does Python's `in`
- | operator use for lists? For sets?

Resources Needed:

- Numbered cards (8 students) for physical demo
- Slide deck with trace tables and pseudocode
- Worksheet 3B (one per student)
- Calculators for $\log_2$ calculations (Secure tier)
- Mini whiteboards for plenary game

WORKSHEET 3A — LINEAR SEARCH

Name: _____ Class: _____ Date: _____

Learning Objective: I can trace a linear search algorithm and explain its best, average, and worst case.

Bilingual Support:

- linear search = 线性搜索 (xiàn xìng sōu suǒ)
- element = 元素 (yuán sù)
- index = 索引 (suǒ yǐn)
- target = 目标 (mù biāo)
- trace table = 追踪表 (zhuī zōng biǎo)
- found = 找到 (zhǎo dào)
- comparison = 比较 (bǐ jiào)

SECTION A: Trace Table Practice [8 marks]

A linear search is performed to find the target value 42 in this list:

[15, 28, 42, 56, 71, 89]

Complete the trace table:

Step	Index	List[Index]	Target	Found	Action
Init	0	—	42	FALSE	Start search
1	—	—	42	—	—
2	—	—	42	—	—
3	—	—	42	—	—

How many comparisons were needed? _____ [1]

SECTION B: Write the Pseudocode [6 marks]

Write pseudocode for a linear search that:

- Has a list: [5, 12, 8, 3, 15, 9]
- Searches for the target value 15
- Outputs "Found" and the position if found
- Outputs "Not found" if not found

Hint: You will need WHILE, IF, and SET.

 [1]

 [1]

 [1]

 [1]

 [1]

 [1]

 [1]

SECTION C: Exam-Style Questions [6 marks]

21. A list contains 1000 items. What is the maximum number of comparisons that linear search would need to make? [1 mark]
22. Explain why this is called the "worst case" for linear search. [2 marks]
23. Give ONE situation where linear search is the best choice. [1 mark]
24. Complete this sentence: Linear search checks each element _____ until the target is _____ or the _____ of the list is reached. [2 marks]

WORKSHEET 3B — BINARY SEARCH & COMPARISON

Name: _____ Class: _____ Date: _____

Learning Objective: I can trace a binary search and compare it with linear search.

Bilingual Support:

- binary search = 二分搜索 (èr fēn sōu suǒ)
- sorted = 已排序 (yǐ pái xù)
- middle element = 中间元素 (zhōng jiān yuán sù)
- midpoint = 中点 (zhōng diǎn)
- divide and conquer = 分治法 (fēn zhì fǎ)
- efficiency = 效率 (xiào lǜ)
- comparison = 比较 (bǐ jiào)

SECTION A: Binary Search Trace Table [8 marks]

A binary search is performed on this sorted list to find Target = 35:

[10, 20, 25, 30, 35, 40, 50, 60]

(Index: 0 1 2 3 4 5 6 7)

Complete the trace table:

Step	Low	High	Mid	List[Mid]	Comparison	Action
Init	0	7	—	—	—	Start
1	—	—	—	—	30 < 35	Low = Mid + 1
2	—	—	—	—	—	—
3	—	—	—	—	—	—

How many comparisons were needed? _____ [1]

How many would linear search need? _____ [1]

SECTION B: Comparison Questions [6 marks]

25. State ONE requirement for binary search to work. [1 mark]

_____ [1]

26. A list has 64 items. In the worst case, how many comparisons for:

(a) Linear search? _____ [1]

(b) Binary search? _____ [1]

(Show your working for binary search)

27. Explain why binary search is generally faster than linear search for large sorted lists. [2 marks]

28. Give ONE reason why a programmer might still choose linear search instead of binary search. [1 mark]

SECTION C: Exam-Style Question [6 marks]

A programmer needs to search for a student's record in a database of 10,000 student records. The records are sorted by student ID number.

(a) Name the most efficient search algorithm for this task. [1 mark]

(b) Explain how this algorithm works. [3 marks]

(c) The records were accidentally saved in random order. Explain how this affects the choice of search algorithm. [2 marks]

ANSWER KEY — WEEK 3

WORKSHEET 3A:

SECTION A:

Step	Index	List[Index]	Target	Found	Action
Init	0	—	42	FALSE	Start search

1	0	15	42	FALSE	Increment
2	1	28	42	FALSE	Increment
3	2	42	42	TRUE	Found!

How many comparisons: 3 [1]

SECTION B:

Accept any logically correct pseudocode. Example answer:

SET List TO [5, 12, 8, 3, 15, 9] [1]

SET Target TO 15

SET Found TO FALSE [1]

SET Index TO 0

WHILE Found == FALSE AND Index < 6 DO [1]

IF List[Index] == Target THEN [1]

SET Found TO TRUE [1]

OUTPUT "Found at position " + Index [1]

ELSE

SET Index TO Index + 1 [1]

ENDIF

ENDWHILE

IF Found == FALSE THEN

OUTPUT "Not found" [1]

ENDIF

SECTION C:

29. 1000 comparisons [1]

30. It is called the worst case because the target is the last item checked [1] or not in the list at all [1], meaning every item must be compared.

31. Any valid situation: when the list is very small [1], when the list is not sorted [1], when searching an unsorted linked structure [1].

32. in order [1], found [1], end [1]

WORKSHEET 3B:

SECTION A:

Step	Low	High	Mid	List[Mid]	Comparison	Action
Init	0	7	—	—	—	Start
1	0	7	3	30	30 < 35	Low = Mid + 1
2	4	7	5	40	40 > 35	High = Mid - 1
3	4	4	4	35	35 == 35	Found!

Comparisons needed (binary): 3 [1]

Comparisons needed (linear): 5 [1]

SECTION B:

1. The list must be sorted [1]

33. (a) 64 [1] (b) 6 [1] (working: $\log_2 64 = 6$, or $2^6 = 64$)

34. Binary search divides the list in half each time [1], so it eliminates half the remaining items with each comparison, making it much faster for large lists [1].

35. The list is not sorted [1], or the list is very small so the difference is negligible [1], or it is simpler to implement [1].

SECTION C:

(a) Binary search [1]

(b) Binary search requires the list to be sorted [1]. It repeatedly finds the middle element [1] and compares it with the target. If the middle is not the target, it discards half the list and repeats [1].

(c) Binary search will NOT work on unsorted data [1]. The programmer must either sort the data first (which takes time) or use linear search instead [1].

WEEK 4: PYTHON — VARIABLES, DATA TYPES, INPUT/OUTPUT

Overview: This week transitions from pseudocode and flowcharts to actual Python programming. Students recap `print()` and variables from Y9S1, then learn the four essential data types (string, integer, float, boolean), variable naming rules, type checking with `type()`, and the `input()` function with type conversion. The week culminates in students writing a complete interactive program using f-strings for formatted output. This is a critical foundational week — all future programming depends on mastering these concepts.

Syllabus Ref: CIE 0478 Topic 2.2 (Data Types) & Topic 2.3 (Programming)

LESSON 4.1 — DATA TYPES & VARIABLES

Topic: Python Data Types, Variable Naming Rules, `type()` Function

Duration: 40 minutes

Lesson Type: Practical Programming (Thonny IDE)

Syllabus Ref: CIE 0478 Topic 2.2

Learning Objective:

By the end of this lesson, all students will be able to declare variables

of four data types (string, integer, float, boolean), follow Python naming conventions, and use the type() function to check variable types.

Key Vocabulary:

- data type = 数据类型 (shù jù lèi xíng)
- string = 字符串 (zì fú chuàn)
- integer = 整数 (zhěng shù)
- float = 浮点数 (fú diǎn shù)
- boolean = 布尔值 (bù ěr zhí)
- variable = 变量 (biàn liàng)
- declare = 声明 (shēng míng)
- initialise = 初始化 (chū shǐ huà)
- naming convention = 命名规范 (mìng míng guī fàn)
- camelCase = 驼峰命名法 (tuó fēng mìng míng fǎ)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | Display on board:
- | name = "Wei Chen"
- | age = 15
- | height = 1.75
- | is_student = True
- | Questions: "How many variables? What values do they store?"
- | Do you remember what = means in Python?"
- | Students write answers in notebooks. Quick check with partner.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Recap from Y9S1:
- | - print() displays output
- | - Variables store data using = (assignment, NOT equals!)
- | - input() gets user input (we will review this next lesson)
- | (Slide 2) The Four Essential Data Types:

Type	Stores	Example
str	Text	"Hello"
int	Whole numbers	42, -7
float	Decimals	3.14, -0.5

| | bool | True/False | True, False |
| _____|

| (Slide 3) Live coding in Thonny (teacher projects screen):

| # String (text)

| school_name = "Victoria Park Academy"

| city = 'Shenzhen'

| print(school_name)

| print(type(school_name)) # <class 'str'>

| # Integer (whole number)

| student_count = 150

| year = 2024

| print(type(student_count)) # <class 'int'>

| # Float (decimal number)

| pi = 3.14159

| temperature = 28.5

| print(type(pi)) # <class 'float'>

| # Boolean (True or False)

| is_sunny = True

| has_homework = False

| print(type(is_sunny)) # <class 'bool'>

| Emphasise: Python guesses the type from the value.

| "Hello" → str, 42 → int, 3.14 → float, True → bool

| (Slide 4) Variable Naming Rules:

| ✓ Must start with a letter or underscore

| ✓ Can contain letters, numbers, underscores

| ✓ Case-sensitive (Name and name are different!)

| ✗ Cannot start with a number

| ✗ Cannot contain spaces

| ✗ Cannot use Python keywords (if, for, while, etc.)

| Good names: student_name, total_score, is_valid

| Bad names: 1stname, total score, if

| Python convention: lowercase with underscores (snake_case)

| NOT camelCase (that is for other languages)

12-25 min | **GUIDED PRACTICE**

| Students open Thonny and follow along.

| Exercise 1: Type and run this code. Predict the output first.

```
| name = "Li Hua"
| age = 14
| gpa = 3.85
| is_boarder = True
| print(name, type(name))
| print(age, type(age))
| print(gpa, type(gpa))
| print(is_boarder, type(is_boarder))
```

| Exercise 2: Fix the errors.

```
| 1st_place = "Gold"    # Error?
| class = "Year 9"      # Error?
| my name = "Tom"       # Error?
| SCORE = 95            # Works but not convention
```

| Students identify and fix each one.

| Exercise 3: Create variables about yourself:

```
| my_name = "your name"
| my_age = your age
| my_height = your height in metres
| likes_coding = True or False
| Print each variable and its type.
```

25-35 min | INDEPENDENT PRACTICE

```
| Worksheet 4A — Section A: Identify the data type
| Section B: Fix variable naming errors
| Section C: Write a program declaring variables
| (Worksheet 4A distributed — practical on Thonny after written)
```

35-40 min | PLENARY + HOMEWORK

```
| Plenary: "Data Type Speed Quiz"
| Teacher calls out a value, students hold up:
| S = string, I = integer, F = float, B = boolean
| "3.14" → F, "100" → S (in quotes!), 100 → I, "True" → S
| True → B, 0.0 → F, -999 → I
| Homework: Create a "Student Profile" program in Thonny.
| Declare at least 6 variables about a fictional student
| (use a Chinese name). Include all 4 data types. Print each
| variable with its type(). Save as student_profile.py
```

EAL Support:

- Colour-coded data type cards: str (blue), int (green), float (orange), bool (purple) — students hold up matching card during quiz
- Bilingual annotations in code comments:
age = 15 # integer 整数
- Sentence frame: "The variable _____ stores a _____ because _____."
- Word bank with data types in English and Chinese on desk
- Pair programming for Thonny exercises (one types, one checks)

Differentiation:

Emerging | Pre-written code with blanks to fill in.

- | Only work with str and int types initially.
- | Matching activity: match value to data type.

Developing | Write code independently in Thonny.

- | Complete all worksheet sections.
- | Identify naming rule violations.

Secure | Extension: Research and use type casting:

- | int("42") → 42, str(42) → "42", float(3) → 3.0
- | bool(1) → True, bool(0) → False, bool("") → False
- | Write a program that demonstrates all type conversions.

Resources Needed:

- Projector for live coding demonstration
- Thonny IDE on all student computers
- Worksheet 4A (one per student)
- Data type colour cards (S/I/F/B) for plenary
- Python keyword reference sheet

LESSON 4.2 — INPUT, TYPE CONVERSION & F-STRINGS

Topic: input() Function, Type Conversion, String Concatenation, f-strings

Duration: 40 minutes

Lesson Type: Practical Programming (Thonny IDE)

Syllabus Ref: CIE 0478 Topic 2.3

Learning Objective:

By the end of this lesson, all students will be able to use input() to get user data, convert between data types, concatenate strings, and use f-strings for formatted output.

Key Vocabulary:

- input() = 输入函数 (shū rù hán shù)

- type conversion = 类型转换 (lèi xíng zhuǎn huàn)
- cast = 强制转换 (qiáng zhì zhuǎn huàn)
- concatenate = 连接 (lián jiē)
- f-string = 格式化字符串 (gé shì huà zì fú chuàn)
- format = 格式化 (gé shì huà)
- prompt = 提示 (tí shì)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Without running it, predict the output:"
- | `age = input("How old are you? ")`
- | `print(type(age))`
- | `next_year = age + 1`
- | `print(next_year)`
- | "Will this work? Why or why not?"
- | Students write prediction. This sets up the type conversion problem that today's lesson solves.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) The input() function recap:
 - | - Always returns a STRING, even if the user types a number!
 - | - This is a very common source of errors.
- | Live coding demonstration:
 - | `age = input("Enter your age: ")`
 - | `print(age)` # Works fine
 - | `print(type(age))` # <class 'str'> ← surprise!
 - | `print(age + 5)` # ERROR! Cannot add str + int
- | (Slide 2) Type Conversion (Casting):

Function	Converts to	Example
<code>int()</code>	Integer	<code>int("15")</code> → 15
<code>float()</code>	Float	<code>float("3.5")</code> → 3.5
<code>str()</code>	String	<code>str(42)</code> → "42"
- | Fixed code:
 - | `age = input("Enter your age: ")`
 - | `age = int(age)` # Convert string to integer!
 - | `print(type(age))` # <class 'int'>
 - | `print(age + 5)` # Works! → 20 (if input was 15)
- | Shortcut (do both in one line):

```
| age = int(input("Enter your age: "))
| (Slide 3) String Concatenation:
| Joining strings together with +
| name = "Zhang Wei"
| print("Hello, " + name + "!")
| # Output: Hello, Zhang Wei!
| Problem: Cannot mix strings and numbers!
| age = 15
| print("You are " + age + " years old") # ERROR!
| Solution 1: Convert number to string
| print("You are " + str(age) + " years old")
| Solution 2 (BETTER): Use f-strings
| print(f"You are {age} years old")
| (Slide 4) f-strings (formatted string literals):
| - Put f before the opening quote
| - Put variables inside curly braces { }
| - Python replaces {variable} with its value
| Examples:
| name = "Mei Ling"
| score = 95
| print(f"Congratulations, {name}! You scored {score}.")
| # Output: Congratulations, Mei Ling! You scored 95.
| f-strings make code much more readable!
```

12-25 min | **GUIDED PRACTICE**

```
| Students follow in Thonny.
| Exercise 1: Fix the type errors.
| # Broken code:
| num1 = input("Enter first number: ")
| num2 = input("Enter second number: ")
| total = num1 + num2
| print(f"Total: {total}")
| # What happens? (Concatenation: "5" + "3" = "53")
| # Fix: Convert to int!
| num1 = int(input("Enter first number: "))
| num2 = int(input("Enter second number: "))
| total = num1 + num2
```

- | `print(f"Total: {total}")`
- | Exercise 2: Create a greeting program.
- | `name = input("What is your name? ")`
- | `age = int(input("How old are you? "))`
- | `print(f"Hello, {name}! Next year you will be {age + 1}.")`
- | Exercise 3: Create a simple calculator for two numbers.
- | (Ask for two numbers, display sum, difference, product)
- | Students work in pairs.

25-35 min | **INDEPENDENT PRACTICE**

- | Worksheet 4B — Section A: Type conversion exercises
- | Section B: Fix broken code
- | Section C: Write a complete program (exam style)
- | Practical: Students then implement in Thonny
- | (Worksheet 4B distributed)

35-40 min | **PLENARY + HOMEWORK**

- | Plenary: "Fix It Fast!"
- | Display broken code on board. First student to identify the error and explain the fix wins a point.
- | Code 1: `print("You are" + 15)`
- | Code 2: `age = int(input("Age: ")) + print(age)`
- | Code 3: `print(f"Hello {name}")` # name not defined
- | Homework: Complete the "Personal Details" program:
- | - Ask for name (string), age (integer), height in metres (float)
- | - Use f-strings to display: "Hello [name], you are [age] years old and [height]m tall."
- | - Calculate and display height in cm (multiply by 100)
- | Save as `personal_details.py`

EAL Support:

- Annotated code examples showing the "before and after" of type conversion
- Sentence frame: "input() always returns a _____, so I need to use _____() to convert it to a _____."
- Visual reminder poster: "input() = str! Convert if you need numbers!"
- f-string template: `print(f"text {variable} more text")`
- Pair programming for all Thonny exercises

Differentiation:

Emerging | Fill-in-the-blanks code worksheet.

| Only use int() conversion (not float).

| Pre-written f-string templates.

Developing | Write complete programs independently.

| Use int(), float(), and str() conversions.

| Write own f-strings.

Secure | Extension: Write a program that asks for a price in RMB,

| converts to USD (rate: 1 USD = 7.24 RMB), and displays

| both with 2 decimal places using f-strings:

| print(f"RMB: {rmb:.2f}")

| Add HKD conversion too (1 HKD = 0.92 RMB).

Resources Needed:

- Thonny IDE on all computers

- Projector for live coding

- Worksheet 4B (one per student)

- "Common Errors" reference sheet

- f-string syntax poster

WORKSHEET 4A — PYTHON DATA TYPES & VARIABLES

Name: _____ Class: _____ Date: _____

Learning Objective: I can identify Python data types, follow variable naming rules, and use the type() function.

Bilingual Support:

- string (str) = 字符串 (zì fú chuàn)

- integer (int) = 整数 (zhěng shù)

- float = 浮点数 (fú diǎn shù)

- boolean (bool) = 布尔值 (bù ěr zhí)

- variable = 变量 (biàn liàng)

- type() = 类型检查 (lèi xíng jiǎn chá)

SECTION A: Identify the Data Type [8 marks]

Write str, int, float, or bool for each value:

36. "Shenzhen" _____ [1]

37. 2024 _____ [1]

38. 3.14159 _____ [1]

4. True _____ **[1]**

39. "True" _____ [1]

40. -15.5 _____ [1]

41.0 _____ [1]

8. False _____ [1]

SECTION B: Variable Naming [6 marks]

Each variable name below has a problem. Identify the error and fix it.

42. 1st_student = "Li Wei"

Error: _____ [1]

Fix: _____ [1]

43. my name = "Chen Hao"

Error: _____ [1]

Fix: _____ [1]

44. class = "Year 9"

Error: _____ [1]

Fix: _____ [1]

SECTION C: Python Program [6 marks]

45. Write a Python program that creates variables for a fictional student with a Chinese name. Include:

- student_name (string)
- student_age (integer)
- student_gpa (float, e.g., 3.85)
- is_enrolled (boolean)

Print each variable and its type using type().

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

_____ [1]

WORKSHEET 4B — INPUT, TYPE CONVERSION & F-STRINGS

Name: _____ Class: _____ Date: _____

Learning Objective: I can use input(), convert data types, and use f-strings for formatted output.

Bilingual Support:

- input() = 输入 (shū rù)
- type conversion = 类型转换 (lèi xíng zhuǎn huàn)
- int() = 转整数 (zhuǎn zhěng shù)
- float() = 转浮点数 (zhuǎn fú diǎn shù)
- str() = 转字符串 (zhuǎn zì fú chuàn)
- f-string = 格式化字符串 (gé shì huà zì fú chuàn)

SECTION A: Type Conversion Practice [6 marks]

Predict the output of each code snippet:

46. age = int("16")

print(age + 4)

Output: _____ [1]

47. price = float("25.99")

print(price * 2)

Output: _____ [1]

48. number = 42

print("The number is " + str(number))

Output: _____ [1]

49. num1 = int(input("Enter: ")) # User enters: 8

num2 = int(input("Enter: ")) # User enters: 3

print(num1 // num2)

Output: _____ [1]

50. x = "10"

y = 5

print(int(x) + y)

Output: _____ [1]

51. name = "Wang"

age = 15

print(f"My name is {name} and I am {age}")

Output: _____ [1]

SECTION B: Find and Fix the Errors [6 marks]

Each program has an error. Identify it and write the corrected version.

7. Program:

age = input("How old are you? ")

print(f"Next year you will be {age + 1}")

Error: _____ [1]

Fixed code:

_____ [1]

8. Program:

```
num1 = input("First number: ")
num2 = input("Second number: ")
print(f"The sum is {num1 + num2}")
```

Error: _____ [1]

Fixed code:

_____ [1]

9. Program:

```
height_cm = 175
print("Your height is " + height_cm + " cm")
```

Error: _____ [1]

Fixed code:

_____ [1]

SECTION C: Write a Complete Program [8 marks]

10. Write a Python program that:

- (a) Asks the user for their name [1]
- (b) Asks for their birth year (as an integer) [1]
- (c) Calculates their age in 2024 [1]
- (d) Displays: "Hello [name], you were born in [year] and you
are approximately [age] years old." using an f-string [1]
- (e) Has appropriate comments in English [1]

ANSWER KEY — WEEK 4

WORKSHEET 4A:

SECTION A:

- 52. str [1]
- 53. int [1]
- 54. float [1]
- 55. bool [1]
- 56. str [1] (because it is in quotes!)
- 57. float [1]
- 58. int [1]

59.bool [1]

SECTION B:

9. Error: Variable name starts with a number [1]

Fix: first_student = "Li Wei" [1]

10. Error: Variable name contains a space [1]

Fix: my_name = "Chen Hao" [1]

11. Error: "class" is a Python keyword [1]

Fix: student_class = "Year 9" [1] (accept any valid alternative)

SECTION C:

60. Example answer (accept any valid program with all 4 types):

```
student_name = "Liu Mei" [1]
```

```
student_age = 14 [1]
```

```
student_gpa = 3.75 [1]
```

```
is_enrolled = True [1]
```

```
print(student_name, type(student_name)) [1]
```

```
print(student_age, type(student_age)) [1]
```

```
print(student_gpa, type(student_gpa)) [1]
```

```
print(is_enrolled, type(is_enrolled)) [1]
```

WORKSHEET 4B:

SECTION A:

61.20 [1]

62.51.98 [1]

3. The number is 42 [1]

63.2 [1]

64.15 [1]

6. My name is Wang and I am 15 [1]

SECTION B:

65. Error: input() returns a string, cannot add 1 to a string [1]

Fix: age = int(input("How old are you? ")) [1]

(or convert before adding)

66. Error: num1 and num2 are strings, + concatenates them instead of adding [1]

Fix: `num1 = int(input("First number: "))`

`num2 = int(input("Second number: ")) [1]`

67. Error: Cannot concatenate string with integer directly [1]

Fix: `print(f"Your height is {height_cm} cm") [1]`

(or: `print("Your height is " + str(height_cm) + " cm")`)

SECTION C:

10. Marking scheme:

- Use of `input()` for name [1]
- Use of `int(input())` for birth year [1]
- Correct age calculation: `age = 2024 - birth_year` [1]
- Correct f-string with all variables [1]
- At least one comment [1]

Example answer:

`# Get user details`

`name = input("Enter your name: ") [1]`

`birth_year = int(input("Enter your birth year: ")) [1]`

`# Calculate age`

`age = 2024 - birth_year [1]`

`# Display result`

`print(f"Hello {name}, you were born in {birth_year} and you are approximately {age} years old.") [1]`

WEEK 5: OPERATORS

Overview: This week covers the two main categories of operators students need for IGCSE: arithmetic operators (for calculations) and comparison/logical operators (for conditions). Lesson 5.1 focuses on arithmetic including the tricky modulo operator and integer division, with real-world context. Lesson

5.2 introduces comparison and logical operators with truth tables, preparing

students for selection statements in Week 6. Mastery of operators is essential — every program from this point forward will use them.

Syllabus Ref: CIE 0478 Topic 2.3 (Programming — Operators)

LESSON 5.1 — ARITHMETIC OPERATORS

Topic: Arithmetic Operators, Integer Division, Modulo, Order of Operations

Duration: 40 minutes

Lesson Type: Conceptual with Practical Calculations

Syllabus Ref: CIE 0478 Topic 2.3

Learning Objective:

By the end of this lesson, all students will be able to use all seven Python arithmetic operators, understand the difference between float and integer division, explain and apply the modulo operator, and follow the correct order of operations (BODMAS).

Key Vocabulary:

- arithmetic operator = 算术运算符 (suàn shù yùn suàn fú)
- addition = 加法 (jiā fǎ)
- subtraction = 减法 (jiǎn fǎ)
- multiplication = 乘法 (chéng fǎ)
- division = 除法 (chú fǎ)
- integer division = 整数除法 (zhěng shù chú fǎ)
- modulo = 取模/取余 (qǔ mó / qǔ yú)
- exponent = 指数/幂 (zhǐ shù / mì)
- BODMAS = 运算顺序 (yùn suàn shùn xù)
- order of operations = 运算优先级 (yùn suàn yōu xiān jí)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | Mental maths warm-up (display on board):
- | 1. $17 + 8 = ?$
- | 2. $100 - 37 = ?$
- | 3. $15 \times 4 = ?$
- | 4. $48 \div 6 = ?$
- | 5. What is the remainder when 23 is divided by 5?
- | Students write answers. Quick whole-class check.
- | Question 5 introduces the concept of "remainder" — the
- | foundation of the modulo operator.

3-12 min | DIRECT INSTRUCTION

| (Slide 1) Arithmetic Operators in Python:

| Operator | Symbol | Example | Result

| _____|_____|_____|_____

| Add | + | $5 + 3$ | 8

| Subtract | - | $10 - 4$ | 6

| Multiply | * | $6 * 7$ | 42

| Divide | / | $15 / 2$ | 7.5

| Int Div | // | $15 // 2$ | 7

| Modulo | % | $15 \% 2$ | 1

| Power | | 2^3 | 8

| (Slide 2) Division explained — the THREE types:

| Normal division (/) always gives a FLOAT:

| $15 / 2 \rightarrow 7.5$

| $8 / 2 \rightarrow 4.0$ (still a float!)

| Integer division (//) gives the WHOLE NUMBER part:

| $15 // 2 \rightarrow 7$ (2 goes into 15 seven times)

| $17 // 5 \rightarrow 3$ (5 goes into 17 three times)

| Modulo (%) gives the REMAINDER:

| $15 \% 2 \rightarrow 1$ ($15 = 2 \times 7 + 1$)

| $17 \% 5 \rightarrow 2$ ($17 = 5 \times 3 + 2$)

| Together: // and % give you ALL the information from division!

| $17 \div 5 = 3$ remainder 2

| $17 // 5 = 3$

| $17 \% 5 = 2$

| (Slide 3) Modulo — real-world examples:

| Clock: 14:00 in 24-hour clock $\rightarrow 14 \% 12 = 2 \rightarrow 2\text{pm}$

| $25 \% 12 = 1 \rightarrow 1\text{am}$ (next day)

| Odd/Even: $\text{number} \% 2 == 0 \rightarrow \text{even}$

| $\text{number} \% 2 == 1 \rightarrow \text{odd}$

| Grouping: 23 students, 4 per group

| $23 // 4 = 5$ full groups

| $23 \% 4 = 3$ students left over

| (Slide 4) BODMAS (Order of Operations):

| B - Brackets (parentheses) first

| O - Orders (powers, roots)

| D - Division and Multiplication (left to right)

| A - Addition and Subtraction (left to right)

| Example: $3 + 4 * 2$

| NOT $(3+4)*2 = 14 \leftarrow \text{wrong!}$

| Correct: $3 + (4*2) = 3 + 8 = 11$

| In Python: Use brackets to make order clear!

| $(3 + 4) * 2 \rightarrow 14$

| $3 + 4 * 2 \rightarrow 11$

12-25 min | GUIDED PRACTICE

| Exercise 1: Predict, then check in Thonny shell.

| `print(20 / 4)` # Prediction: ?

| `print(20 // 4)` # Prediction: ?

| `print(20 % 4)` # Prediction: ?

| `print(20 / 3)` # Prediction: ?

| `print(20 // 3)` # Prediction: ?

| `print(20 % 3)` # Prediction: ?

| Answer key: 5.0, 5, 0, 6.666..., 6, 2

| Exercise 2: Work out on paper first.

| 1. $27 // 5 = ?$ $27 \% 5 = ?$

| 2. $100 // 7 = ?$ $100 \% 7 = ?$

| 3. $50 // 6 = ?$ $50 \% 6 = ?$

| Exercise 3: BODMAS practice.

| 1. $8 + 3 * 2 = ?$

| 2. $(8 + 3) * 2 = ?$

| 3. $10 - 4 / 2 = ?$

| 4. $(10 - 4) / 2 = ?$

| 5. $2 ** 3 + 4 * 2 = ?$

25-35 min | INDEPENDENT PRACTICE

| Worksheet 5A — Section A: Operator matching

| Section B: Division calculations (`//` and `%`)

| Section C: BODMAS and exam questions

| (Worksheet 5A distributed)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Operator Bingo"

| Teacher calls: "The remainder when 17 is divided by 3!"

| Students call back: " $17 \% 3 = 2$!"

| 5 quick-fire questions.

| Homework: Write 5 real-world problems that use `//` and `%`.

| Example: "A factory packs 43 toys into boxes of 6. How many

| full boxes? How many toys left over?"

| Include the Python code to solve each problem.

EAL Support:

- Visual diagram of division: show $17 \div 5$ with physical objects (e.g., 17 counters grouped into 5s $\rightarrow$ 3 groups, 2 left over)
- BODMAS poster with colour-coded levels (brackets = red, etc.)
- Sentence frame: "_____ divided by _____ equals _____ remainder _____, so // gives _____ and % gives _____."
- Clock face visual for modulo example
- Word bank with operator symbols and names in both languages

Differentiation:

Emerging | Calculator allowed for basic arithmetic.

- | Focus on +, -, *, /, //, % only (no **).
- | Pre-filled tables for division practice.
- | Only complete Section A and B.

Developing | Complete all sections independently.

- | Work out BODMAS problems on paper before checking.
- | Apply to simple programming contexts.

Secure | Extension: Write a program that converts seconds into

- | hours, minutes, and seconds using // and %.
- | Example: 3665 seconds = 1 hour, 1 minute, 5 seconds.
- | Also: Research and explain the difference between
- | Python's % and mathematical modulo for negative numbers.

Resources Needed:

- Physical counters for division demonstration
- Slide deck with operator table
- Worksheet 5A (one per student)
- BODMAS poster
- Thonny for guided practice checking
- Clock face visual aid

LESSON 5.2 — COMPARISON & LOGICAL OPERATORS

Topic: Comparison Operators, Logical Operators, Truth Tables

Duration: 40 minutes

Lesson Type: Conceptual with Practical Application

Syllabus Ref: CIE 0478 Topic 2.3

Learning Objective:

By the end of this lesson, all students will be able to use all six comparison operators, apply AND, OR, and NOT logical operators, complete

a truth table, and build a compound condition in Python.

Key Vocabulary:

- comparison operator = 比较运算符 (bǐ jiào yùn suàn fú)
- logical operator = 逻辑运算符 (luó jí yùn suàn fú)
- equal to = 等于 (děng yú)
- not equal to = 不等于 (bù děng yú)
- greater than = 大于 (dà yú)
- less than = 小于 (xiǎo yú)
- AND = 与/并且 (yǔ / bìng qiě)
- OR = 或/或者 (huò / huò zhě)
- NOT = 非/不 (fēi / bù)
- truth table = 真值表 (zhēn zhí biǎo)
- compound condition = 复合条件 (fù hé tiáo jiàn)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "True or False?" (display on board):
- | 1. 5 is greater than 3
- | 2. 10 is equal to 7
- | 3. 4 is less than or equal to 4
- | 4. "Hello" is the same as "hello"
- | Students write T/F for each. Whole-class check.
- | This activates prior knowledge of comparison concepts.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Comparison Operators:
- | Operator | Symbol | Meaning | Example | Result
- | ————+————+————+————+————+————
- | Equal | == | is equal to | 5 == 5 | True
- | Not equal | != | is not equal to | 5 != 3 | True
- | Greater | > | greater than | 5 > 3 | True
- | Less | < | less than | 5 < 3 | False
- | >= | >= | greater or equal | 5 >= 5 | True
- | <= | <= | less or equal | 5 <= 4 | False
- | CRITICAL: == is COMPARISON (asks a question)
- | = is ASSIGNMENT (gives a value)
- | This is the #1 mistake in Python!

| (Slide 2) Live coding in Thonny:

```
| print(10 == 10)  # True
| print(10 == 5)   # False
| print(10 != 5)   # True
| print(7 > 3)     # True
| print(7 < 3)     # False
| score = 85
| print(score >= 60) # True (passed!)
| print(score >= 90) # False (not an A)
```

| (Slide 3) Logical Operators:

| AND - both conditions must be True
| OR - at least one condition must be True
| NOT - reverses the result

| Examples:

```
| age = 16
| has_id = True
| print(age >= 18 AND has_id == True) # False
| score = 75
| print(score < 60 OR score > 90)    # False
| is_raining = True
| print(NOT is_raining)              # False
```

| (Slide 4) Truth Tables:

| AND Truth Table: OR Truth Table:

A	B	A AND B	A	B	A OR B
---	---	---------	---	---	--------

—	—	—	—	—	—
---	---	---	---	---	---

True	True	True	True	True	True
------	------	------	------	------	------

True	False	False	True	False	True
------	-------	-------	------	-------	------

False	True	False	False	True	True
-------	------	-------	-------	------	------

False	False	False	False	False	False
-------	-------	-------	-------	-------	-------

| NOT Truth Table:

A	NOT A
---	-------

—	—
---	---

True	False
------	-------

False	True
-------	------

12-25 min | GUIDED PRACTICE

| Exercise 1: Complete truth tables on mini whiteboards.

| Given: age = 20, has_ticket = True, is_vip = False

| 1. age >= 18 AND has_ticket

| 2. age >= 18 AND is_vip

| 3. age < 18 OR has_ticket

| 4. NOT is_vip

| 5. age >= 18 AND (has_ticket OR is_vip)

| Answers: True, False, True, True, True

| Exercise 2: In Thonny, test these expressions.

| x = 10

| y = 20

| print(x > 5 AND y < 30) # True

| print(x > 15 OR y < 30) # True

| print(NOT(x == y)) # True

| Exercise 3: "Build a Condition"

| Teacher describes a scenario, students write the condition.

| "A student passes if they score at least 60 AND

| attend at least 80% of classes."

| → score >= 60 AND attendance >= 80

| "You get a discount if you are a senior (age >= 65)

| OR a student (age < 25)."

| → age >= 65 OR age < 25

25-35 min | INDEPENDENT PRACTICE

| Worksheet 5B — Section A: Comparison operator practice

| Section B: Truth tables

| Section C: Compound conditions

| (Worksheet 5B distributed)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Logic Puzzle"

| Given three boolean variables, determine the result.

| a = 5 > 3 # True

| b = 10 == 8 # False

| c = NOT(4 > 7) # True

| What is: a AND b OR c?

| (Answer: True, because True AND False = False,

| then False OR True = True)

| Homework: Create 5 compound conditions for real-world scenarios.

| Write both the English description AND the Python expression.

| Example: "You can drive if age $\geq$ 18 AND has_license == True"

EAL Support:

- == vs = visual poster with LARGE symbols and colour coding

- Truth table templates pre-printed

- Sentence frames for conditions:

"The condition _____ is _____ because _____."

"A AND B is True only when both A _____ and B _____."

- Colour-code: True (green), False (red)

- Logical operator mnemonics:

AND = "both must be yes" (两个都必须)

OR = "at least one yes" (至少一个)

NOT = "opposite" (相反)

Differentiation:

Emerging | Focus on ==, !=, >, < only (not $\geq$, $\leq$ initially).

| Pre-completed truth tables — only fill in 2 rows.

| Match-up: match condition to True/False result.

| Only complete Section A.

Developing | Complete all truth table rows.

| Write compound conditions from descriptions.

| Use all operators confidently.

Secure | Extension: Write a "compound condition checker" program

| that asks the user for values and evaluates complex

| conditions with multiple AND/OR/NOT combinations.

| Research: What is short-circuit evaluation in Python?

| Demonstrate with: True OR (1/0) — why no error?

Resources Needed:

- Slide deck with operator tables and truth tables

- Worksheet 5B (one per student)

- Mini whiteboards for guided practice

- Thonny for live coding

- Truth table templates

- == vs = visual reminder poster

WORKSHEET 5A — ARITHMETIC OPERATORS

Name: _____ Class: _____ Date: _____

Learning Objective: I can use arithmetic operators, understand // and %, and apply BODMAS.

Bilingual Support:

- addition (+) = 加法 (jiā fǎ)
- subtraction (-) = 减法 (jiǎn fǎ)
- multiplication (*) = 乘法 (chéng fǎ)
- division (/) = 除法 (chú fǎ)
- integer division (//) = 整数除法 (zhěng shù chú fǎ)
- modulo (%) = 取模/取余 (qǔ mó / qǔ yú)
- exponent (**) = 幂运算 (mì yùn suàn)
- BODMAS = 运算顺序 (yùn suàn shùn xù)

SECTION A: Operator Matching [6 marks]

Match each description to the correct Python operator.

- | | |
|------------------------------|--------------|
| 1. Addition | a. // |
| 2. Integer division | b. % |
| 3. Modulo (remainder) | c. ** |
| 4. Exponent (power) | d. + |
| 5. Normal division | e. - |
| 6. Subtraction | f. / |

SECTION B: Division Practice [8 marks]

For each calculation, give the result of /, //, and %.

7. For 17 divided by 5:

$17 / 5 = \underline{\hspace{2cm}}$ $17 // 5 = \underline{\hspace{2cm}}$ $17 \% 5 = \underline{\hspace{2cm}}$ [2]

8. For 30 divided by 4:

$30 / 4 = \underline{\hspace{2cm}}$ $30 // 4 = \underline{\hspace{2cm}}$ $30 \% 4 = \underline{\hspace{2cm}}$ [2]

9. For 100 divided by 8:

$100 / 8 = \underline{\hspace{2cm}}$ $100 // 8 = \underline{\hspace{2cm}}$ $100 \% 8 = \underline{\hspace{2cm}}$ [2]

10. For 9 divided by 3:

$$9 / 3 = \underline{\hspace{2cm}} \quad 9 // 3 = \underline{\hspace{2cm}} \quad 9 \% 3 = \underline{\hspace{2cm}} \quad [2]$$

SECTION C: BODMAS & Exam Questions [6 marks]

Calculate each expression. Show your working.

$$68. 10 + 6 * 2 = \underline{\hspace{2cm}} \quad [1]$$

$$69. (10 + 6) * 2 = \underline{\hspace{2cm}} \quad [1]$$

$$70. 20 / 4 + 3 * 2 = \underline{\hspace{2cm}} \quad [1]$$

$$71. 2 ** 3 + 5 // 2 = \underline{\hspace{2cm}} \quad [1]$$

15. A school has 137 students. Each bus holds 45 students.

(a) How many full buses? $\underline{\hspace{2cm}}$ [1]

(b) How many students on the last (not full) bus? $\underline{\hspace{2cm}}$ [1]

WORKSHEET 5B — COMPARISON & LOGICAL OPERATORS

Name: $\underline{\hspace{2cm}}$ Class: $\underline{\hspace{2cm}}$ Date: $\underline{\hspace{2cm}}$

Learning Objective: I can use comparison and logical operators and complete truth tables.

Bilingual Support:

- equal to (==) = 等于 (děng yú)
- not equal (!=) = 不等于 (bù děng yú)
- greater than (>) = 大于 (dà yú)
- less than (<) = 小于 (xiǎo yú)
- AND = 并且 (bìng qiě)
- OR = 或者 (huò zhě)
- NOT = 非 (fēi)
- truth table = 真值表 (zhēn zhí biǎo)

SECTION A: Comparison Operators [6 marks]

Predict the output (True or False):

$$72. \text{print}(10 == 10) \quad \underline{\hspace{2cm}} \quad [1]$$

$$73. \text{print}(7 != 7) \quad \underline{\hspace{2cm}} \quad [1]$$

$$74. \text{print}(15 > 20) \quad \underline{\hspace{2cm}} \quad [1]$$

$$75. \text{print}(8 <= 8) \quad \underline{\hspace{2cm}} \quad [1]$$

$$76. \text{print}(3 + 5 == 8) \quad \underline{\hspace{2cm}} \quad [1]$$

$$77. \text{print}(10 / 2 == 5.0) \quad \underline{\hspace{2cm}} \quad [1]$$

SECTION B: Truth Tables [8 marks]

7. Complete the AND truth table:

A	B	A AND B	
True	True	_____	[1]
True	False	_____	[1]
False	True	_____	[1]
False	False	_____	[1]

8. Complete the OR truth table:

A	B	A OR B	
True	True	_____	[1]
True	False	_____	[1]
False	True	_____	[1]
False	False	_____	[1]

SECTION C: Compound Conditions [6 marks]

Given: age = 17, has_passed = True, score = 85

9. What is the result of each condition? (True or False)

- (a) age >= 18 _____ [1]
(b) age >= 16 AND has_passed == True _____ [1]
(c) score >= 90 OR has_passed == True _____ [1]
(d) NOT(age < 16) _____ [1]
(e) age >= 16 AND score >= 60 AND has_passed == True _____ [1]
(f) score < 60 OR score > 100 _____ [1]

ANSWER KEY — WEEK 5

WORKSHEET 5A:

SECTION A:

78. d (+) [1]
79. a (//) [1]
80. b (%) [1]
81. c (**) [1]
82. f (/) [1]
83. e (-) [1]

SECTION B:

84. 3.4, 3, 2 [2] (deduct 1 for each error)
85. 7.5, 7, 2 [2]
86. 12.5, 12, 4 [2]

87.3.0, 3, 0 [2]

SECTION C:

88.22 [1] ($6 * 2 = 12$, $10 + 12 = 22$)

89.32 [1] ($10 + 6 = 16$, $16 * 2 = 32$)

90.11.0 [1] ($20 / 4 = 5$, $3 * 2 = 6$, $5 + 6 = 11$)

91.10 [1] ($2 * 3 = 8$, $5 // 2 = 2$, $8 + 2 = 10$)

92.(a) 3 [1] ($137 // 45 = 3$)

(b) 2 [1] ($137 \% 45 = 2$)

WORKSHEET 5B:

SECTION A:

1. True [1]

2. False [1]

3. False [1]

4. True [1]

5. True [1]

6. True [1] ($10 / 2 = 5.0$, $5.0 == 5.0$)

SECTION B:

7. AND truth table:

True, False, False, False [1 each]

8. OR truth table:

True, True, True, False [1 each]

SECTION C:

93.(a) False [1] ($17 \geq 18$ is False)

(b) True [1] (True AND True = True)

(c) True [1] (False OR True = True)

(d) True [1] (NOT(False) = True)

(e) True [1] (True AND True AND True = True)

(f) False [1] (False OR False = False)

WEEK 6: SELECTION (IF STATEMENTS)

Overview: This week brings together all the concepts from Weeks 4 and 5 (variables, data types, input/output, comparison operators, logical operators) to implement selection in Python. Lesson 6.1 covers IF, ELSE, and ELIF with indentation rules, building a grade calculator. Lesson 6.2 covers nested IF statements, complex conditions, and debugging common errors. By the end of this week, students can write programs that make decisions — the most fundamental programming structure after sequence. This directly maps to CIE Topic 2.4 (Selection).

Syllabus Ref: CIE 0478 Topic 2.4 (Selection)

LESSON 6.1 — IF, ELSE, AND ELIF

Topic: Selection Statements, Indentation, Grade Calculator Program

Duration: 40 minutes

Lesson Type: Practical Programming (Thonny IDE)

Syllabus Ref: CIE 0478 Topic 2.4

Learning Objective:

By the end of this lesson, all students will be able to write Python programs using IF, ELSE, and ELIF statements, follow correct indentation rules, use comparison operators in conditions, and build a multi-condition program.

Key Vocabulary:

- selection = 选择结构 (xuǎn zé jié gòu)
- if statement = 条件语句 (tiáo jiàn yǔ jù)
- condition = 条件 (tiáo jiàn)
- else = 否则 (fǒu zé)
- elif = 否则如果 (fǒu zé rú guǒ) / else-if
- indentation = 缩进 (suō jìn)
- block = 代码块 (dài mǎ kuài)
- colon = 冒号 (mào hào)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You are writing a program for a theme park. The rule is:
- | If you are 12 or older, you pay full price (200 RMB).
- | If you are under 12, you pay child price (100 RMB).
- | Write pseudocode for this."
- | Students write for 2 minutes. Collect 2 examples.

| This sets up the IF...ELSE structure.

3-12 min | **DIRECT INSTRUCTION**

| (Slide 1) Selection in Programming:

| "So far, all our programs run line by line from top to bottom (SEQUENCE). Selection lets us CHOOSE which code to run based on a condition."

| Recap from flowcharts: the decision diamond becomes IF.

| (Slide 2) The IF statement:

| if condition:

| # code to run if condition is True

| RULES:

- | 1. The condition must evaluate to True or False
- | 2. Must have a COLON (:) at the end of the if line
- | 3. The code INSIDE the if must be INDENTED (4 spaces)
- | 4. Python uses indentation to know what belongs to the if

| Live coding in Thonny:

| age = 15

| if age >= 18:

| print("You are an adult")

| print("Program finished")

| "What prints?" (Answer: only "Program finished" because

| age >= 18 is False)

| (Slide 3) IF...ELSE:

| if condition:

| # runs if True

| else:

| # runs if False

| Live coding:

| age = 15

| if age >= 18:

| print("You are an adult")

| else:

| print("You are a minor")

| Output: "You are a minor"

| (Slide 4) IF...ELIF...ELSE:

| ELIF = "else if" — check another condition

```
| if condition1:
|     # runs if condition1 is True
| elif condition2:
|     # runs if condition1 is False AND condition2 is True
| else:
|     # runs if ALL conditions are False
| Live coding — Grade Calculator:
| score = int(input("Enter your score (0-100): "))
| if score >= 90:
|     print("Grade: A")
| elif score >= 80:
|     print("Grade: B")
| elif score >= 70:
|     print("Grade: C")
| elif score >= 60:
|     print("Grade: D")
| else:
|     print("Grade: F")
| Key point: Order matters! score=85 matches the FIRST
| true condition (>=80), not >=70.
```

12-25 min | **GUIDED PRACTICE**

```
| Students follow in Thonny.
| Exercise 1: Fix the indentation errors.
| # Broken code:
| score = 75
| if score >= 60:
| print("Pass")
| else:
|     print("Fail")
| Students identify and fix.
| Exercise 2: Complete the age category program.
| age = int(input("Enter your age: "))
| if age < 13:
|     print("Child")
| elif age < 20:
|     print("Teenager")
```

```
| elif _____:  
|     print("Adult")  
| else:  
|     print("Senior")  
| Answer: age < 65 (or age >= 20 and age < 65)  
| Exercise 3: Build a simple password checker.  
| password = input("Enter password: ")  
| if password == "Shenzhen2024":  
|     print("Access granted")  
| else:  
|     print("Access denied")
```

25-35 min | INDEPENDENT PRACTICE

```
| Worksheet 6A — Section A: Identify output of if statements  
| Section B: Fix broken code (indentation, == vs =)  
| Section C: Write a grade calculator program  
| (Worksheet 6A distributed)
```

35-40 min | PLENARY + HOMEWORK

```
| Plenary: "Code Trace"  
| Display code on board. Students trace through mentally and  
| predict the output.  
| score = 72  
| if score >= 90:  
|     grade = "A"  
| elif score >= 80:  
|     grade = "B"  
| elif score >= 70:  
|     grade = "C"  
| else:  
|     grade = "F"  
| print(f"Your grade is {grade}")  
| Answer: "Your grade is C"  
| Homework: Extend the grade calculator program:  
| - Ask for the student's name  
| - Ask for their score (0-100)  
| - Display: "[Name], your score is [score] and your grade is [grade]"  
| - Add validation: if score < 0 or score > 100, print "Invalid score"
```

| Save as grade_calculator.py

EAL Support:

- Visual indentation guide: draw vertical lines to show indentation levels
- Colour-code: if (blue), elif (green), else (orange)
- Sentence frame: "If _____ is True, then _____. Otherwise, _____."
- Side-by-side pseudocode and Python comparison on slide
- Indentation error examples shown with visual markers
- == vs = poster: == asks "are they equal?", = says "store this value"

Differentiation:

Emerging | Fill-in-the-blanks code with indentation guides.

- | Only use IF and ELSE (no ELIF).
- | Pre-written condition templates.
- | Complete Section A only.

Developing | Write complete IF...ELIF...ELSE programs.

- | Fix indentation and syntax errors.
- | Complete all worksheet sections.

Secure | Extension: Write a BMI calculator that:

- | - Asks for weight (kg) and height (m)
- | - Calculates BMI = weight / (height ** 2)
- | - Outputs the BMI category:
 - | Under 18.5: Underweight
 - | 18.5-24.9: Normal
 - | 25-29.9: Overweight
 - | 30+: Obese
- | Use nested if or chained elif.

Resources Needed:

- Thonny IDE on all computers
- Projector for live coding
- Worksheet 6A (one per student)
- Indentation guides (printed)
- == vs = reminder poster
- Colour-coded keyword cards

LESSON 6.2 — NESTED IF AND COMPLEX CONDITIONS

Topic: Nested IF Statements, Complex Conditions, Common Errors, Login System

Duration: 40 minutes

Lesson Type: Practical Programming with Debugging Focus

Syllabus Ref: CIE 0478 Topic 2.4

Learning Objective:

By the end of this lesson, all students will be able to write nested IF statements, combine conditions with AND/OR, identify and fix common selection errors (indentation, == vs =, missing colon), and build a multi-condition login system.

Key Vocabulary:

- nested if = 嵌套条件 (qiàn tào tiáo jiàn)
- complex condition = 复合条件 (fù hé tiáo jiàn)
- debug = 调试 (tiáo shì)
- syntax error = 语法错误 (yǔ fǎ cuò wù)
- logic error = 逻辑错误 (luó jí cuò wù)
- indentation error = 缩进错误 (suō jìn cuò wù)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Debug this code — find 3 errors:"
- | score = 85
- | if score = 100:
- | print("Perfect")
- | elif score >= 80
- | print("Good")
- | else
- | print("Keep trying")
- | Students circle the errors. This sets up today's focus on
- | debugging common mistakes.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Common Errors in Selection:
- | Error 1: Using = instead of ==
- | if score = 100: ← WRONG! This assigns 100 to score
- | if score == 100: ← CORRECT! This compares score to 100
- | Error 2: Missing colon
- | if score >= 60 ← WRONG! Missing :
- | if score >= 60: ← CORRECT!
- | Error 3: Wrong indentation
- | if score >= 60:

```
| print("Pass")    ← WRONG! Must be indented
| if score >= 60:
|     print("Pass") ← CORRECT! (4 spaces)
| Error 4: Forgetting quotes around strings
| if name == Zhang: ← WRONG! Zhang is treated as variable
| if name == "Zhang": ← CORRECT!
```

| (Slide 2) Nested IF Statements:

```
| An IF inside another IF.
| if condition1:
|     if condition2:
|         # runs if BOTH are True
|     else:
|         # runs if condition1=True, condition2=False
| else:
|     # runs if condition1 is False
```

| Example: Check age AND ticket for a concert.

```
| age = int(input("Enter age: "))
| has_ticket = input("Do you have a ticket? (yes/no): ")
| if age >= 16:
|     if has_ticket == "yes":
|         print("Enjoy the concert!")
|     else:
|         print("You need a ticket.")
| else:
|     print("You must be 16 or older.")
```

| (Slide 3) Complex Conditions with AND/OR:

| Instead of nested IFs, we can often combine conditions:

```
| # Using AND (both must be True)
| if age >= 16 AND has_ticket == "yes":
|     print("Enjoy the concert!")
| elif age < 16:
|     print("You must be 16 or older.")
| else:
|     print("You need a ticket.")
| # Using OR (at least one must be True)
| if day == "Saturday" OR day == "Sunday":
```

```
| print("It is the weekend!")
| (Slide 4) Nested IF vs AND — when to use which:
| Use AND when: both conditions are checked at the same level
| Use nested IF when: the second check only matters if the
| first condition is True
```

12-25 min | **GUIDED PRACTICE**

```
| Exercise 1: Convert nested IF to AND condition.
| # Nested version:
| if score >= 60:
|     if attendance >= 80:
|         print("Pass with good attendance")
| # Convert to:
| if score >= 60 AND attendance >= 80:
|     print("Pass with good attendance")
| Exercise 2: "Find the errors" — students get printed code
| with 5 common errors. Circle and fix each one.
| Exercise 3: Build a login system (guided).
| username = input("Username: ")
| password = input("Password: ")
| if username == "admin":
|     if password == "secret123":
|         print("Login successful!")
|     else:
|         print("Wrong password")
| else:
|     print("User not found")
| Then refactor using AND:
| if username == "admin" AND password == "secret123":
|     print("Login successful!")
| else:
|     print("Login failed")
```

25-35 min | **INDEPENDENT PRACTICE**

```
| Worksheet 6B — Section A: Find and fix errors
| Section B: Write nested IF programs
| Section C: Complex conditions exam question
| Practical: Implement in Thonny
```


| (Worksheet 6B distributed)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Code Review"

| Display a student's (anonymised) solution to the login system.

| Class evaluates: Is the logic correct? Is the indentation

| correct? Could it be improved?

| Homework: Build a "Theme Park Ride" eligibility checker:

| - Ask for age and height

| - Rules: Must be age ≥ 10 AND height $\geq 130\text{cm}$

| - If too young: "You are too young for this ride."

| - If too short: "You are too short for this ride."

| - If both OK: "Enjoy the ride!"

| Use nested IF or AND condition (student's choice).

| Save as theme_park.py

EAL Support:

- Error highlighting: use coloured highlighters to mark different error types (red for syntax, yellow for logic, green for indentation)

- Sentence frames for nested logic:

"First, check if _____. If yes, then check if _____."

"Both conditions must be true: _____ AND _____."

- Visual nesting diagram showing indentation levels with colours

- Side-by-side: nested IF vs AND version of same logic

- Common errors reference sheet with visual examples

Differentiation:

Emerging | Focus on IF...ELSE only, no nesting.

| Pre-written code with blanks for conditions only.

| Match error type to code example.

| Complete Section A only.

Developing | Write nested IF and AND conditions.

| Find and fix errors independently.

| Complete all worksheet sections.

Secure | Extension: Write a complete "Bank ATM" simulator:

| - Ask for PIN (correct: 1234)

| - If PIN correct, show menu: Check Balance / Withdraw /

| Deposit (use nested if or elif)

| - If PIN wrong, allow 3 attempts before locking

- | - Use compound conditions (AND for PIN + amount validation)
- | Add comments explaining each section.

Resources Needed:

- Thonny IDE on all computers
- Projector for live coding and code review
- Worksheet 6B (one per student)
- "Common Errors" reference sheet
- Coloured highlighters for error-finding activity
- Pre-written buggy code for "Find the Errors" exercise

WORKSHEET 6A — IF, ELSE, AND ELIF

Name: _____ Class: _____ Date: _____

Learning Objective: I can write Python programs using if, elif, and else statements with correct indentation.

Bilingual Support:

- if = 如果 (rú guǒ)
- else = 否则 (fǒu zé)
- elif = 否则如果 (fǒu zé rú guǒ)
- condition = 条件 (tiáo jiàn)
- indentation = 缩进 (suō jìn)
- colon = 冒号 (mào hào)

SECTION A: Predict the Output [6 marks]

For each program, write the exact output.

1.

```
score = 78
if score >= 80:
    print("Excellent")
elif score >= 60:
    print("Good")
else:
    print("Needs work")
```

Output: _____ [1]

2.

```
age = 15
if age >= 18:
    print("Adult")
```

```
else:  
    print("Not an adult")
```

Output: _____ [1]

3.

```
temperature = 35  
if temperature > 30:  
    print("Hot")  
elif temperature > 20:  
    print("Warm")  
elif temperature > 10:  
    print("Cool")  
else:  
    print("Cold")
```

Output: _____ [1]

4.

```
x = 10  
if x == 10:  
    print("Ten")  
if x > 5:  
    print("Greater than five")
```

Output: _____ [1]

(Be careful — there are TWO separate if statements!)

5.

```
password = "hello"  
if password == "secret":  
    print("Welcome")  
else:  
    print("Wrong password")
```

Output: _____ [1]

6.

```
num = 7  
if num % 2 == 0:  
    print("Even")  
else:  
    print("Odd")
```

Output: _____ [1]

SECTION B: Fix the Errors [8 marks]

Each program has at least one error. Find and fix it.

7.

```
age = 16
if age >= 18
    print("Adult")
else:
    print("Minor")
```

Error: _____ [1]

Fixed line: _____ [1]

8.

```
score = 90
if score = 100:
    print("Perfect")
else:
    print("Not perfect")
```

Error: _____ [1]

Fixed line: _____ [1]

9.

```
name = "Li Wei"
if name == Li Wei:
    print("Hello, Li Wei")
```

Error: _____ [1]

Fixed line: _____ [1]

10.

```
temperature = 25
if temperature > 30:
    print("Hot")
elif temperature > 20:
    print("Warm")
else:
    print("Cold")
```

Error: _____ [1]

Fixed line: _____ [1]

SECTION C: Write a Program [6 marks]

11. Write a Python program that:

- (a) Asks the user for their exam score (0-100) [1]
- (b) Uses if, elif, and else to give a grade: [3]
 - 90-100: "Grade A"
 - 80-89: "Grade B"
 - 70-79: "Grade C"
 - 60-69: "Grade D"
 - Below 60: "Grade F"
- (c) Uses an f-string to display: "Your score is [score] and your grade is [grade]" [1]
- (d) Has correct indentation throughout [1]

WORKSHEET 6B — NESTED IF & COMPLEX CONDITIONS

Name: _____ Class: _____ Date: _____

Learning Objective: I can write nested if statements, use AND/OR in conditions, and debug common selection errors.

Bilingual Support:

- nested if = 嵌套条件 (qiàn tào tiáo jiàn)
- complex condition = 复合条件 (fù hé tiáo jiàn)
- debug = 调试 (tiáo shì)
- AND = 并且 (bìng qiě)
- OR = 或者 (huò zhě)
- syntax error = 语法错误 (yǔ fǎ cuò wù)
- logic error = 逻辑错误 (luó jí cuò wù)

SECTION A: Find and Fix the Bugs [8 marks]

Each program has errors. Find them and write the corrected version.

1.

```
username = "admin"
password = "1234"
if username == "admin" and password == "1234"
    print("Access granted")
```

Errors: _____ [2]

Corrected version:

2.

```
age = 20
country = "China"
```

```

if age >= 18:
    if country == "China":
        print("You can vote in China")
    else:
        print("Different voting rules")
else:
    print("Too young")

```

Error: _____ [2]

Corrected version:

SECTION B: Nested IF Practice [6 marks]

3. Complete this program for a cinema ticket system:

```

age = int(input("Enter your age: "))
is_student = input("Are you a student? (yes/no): ")
if age < 13:
    price = 30
else:
    if is_student == "yes":
        price = 50
    else:
        price = 80
print(f"Your ticket price is {price} RMB")

```

(a) What is the price if age = 15 and is_student = "yes"? _____ [1]

(b) What is the price if age = 10 and is_student = "no"? _____ [1]

(c) What is the price if age = 20 and is_student = "no"? _____ [1]

94. Rewrite the cinema program using AND or OR instead of nested if:

_____ [1]

_____ [1]

_____ [1]

SECTION C: Exam-Style Question [6 marks]

95. A student is writing a program to check if a user can register for a sports club. The rules are:

- Must be aged between 12 and 18 (inclusive)
- Must have parental consent (consent = "yes")

(a) Write a single IF condition using AND to check both rules. [2]

if _____:

(b) Write pseudocode for the complete registration check,
including appropriate messages for success and failure. [4]

ANSWER KEY — WEEK 6

WORKSHEET 6A:

SECTION A:

1. Good [1] (78 >= 60, first elif that is true)

2. Not an adult [1] (15 < 18)

3. Hot [1] (35 > 30, first condition is true)

4. Ten [newline] Greater than five [1]

(Both if conditions are checked independently!)

5. Wrong password [1] ("hello" != "secret")

6. Odd [1] (7 % 2 = 1, not 0)

SECTION B:

7. Error: Missing colon after condition [1]

Fixed: if age >= 18: [1]

96. Error: Using = (assignment) instead of == (comparison) [1]

Fixed: if score == 100: [1]

97. Error: "Li Wei" needs quotes — without quotes, Python thinks Li and Wei are variables [1]

Fixed: if name == "Li Wei": [1]

98. Error: Missing indentation on the print("Warm") line [1]

Fixed: Add 4 spaces before print("Warm") [1]

SECTION C:

11. Marking scheme:

- Use of input() and int() conversion [1]
- Correct if/elif/else structure [1]
- All five grade boundaries correct [1]
- Correct order (highest first) [1]
- f-string with both variables [1]
- Correct indentation [1]

Example answer:

```
score = int(input("Enter your score (0-100): ")) [1]
if score >= 90: [1]
    grade = "A" [1]
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(f"Your score is {score} and your grade is {grade}") [1]
(indentation correct throughout [1])
```

WORKSHEET 6B:

SECTION A:

1. Errors:

- Missing colon at end of if line [1]
- "and" should be "AND" in pseudocode, but in Python it is lowercase "and" (accept either: missing colon is the main error) [1]

Corrected:

```
if username == "admin" and password == "1234": [1]
    print("Access granted")
```

99. Error: The inner if block (print("You can vote in China")) is not indented correctly [1]. It should be indented 8 spaces (4 spaces inside the outer if, plus 4 more for the inner if) [1].

Corrected:

```
if age >= 18:
    if country == "China":
```



```

        print("You can vote in China") [1 - now double indented]
    else:
        print("Different voting rules")
    else:
        print("Too young")

```

SECTION B:

100. (a) 50 [1] (age ≥ 13 branch $\rightarrow$ student = yes $\rightarrow$ price = 50)
 (b) 30 [1] (age < 13 branch $\rightarrow$ price = 30)
 (c) 80 [1] (age $\geq 13 \rightarrow$ not student $\rightarrow$ price = 80)

4. Example answer using elif:

```

age = int(input("Enter your age: ")) [1]
is_student = input("Are you a student? (yes/no): ")
if age < 13: [1]
    price = 30
elif is_student == "yes":
    price = 50
else:
    price = 80
print(f"Your ticket price is {price} RMB") [1]

```

SECTION C:

101. (a) if age ≥ 12 AND age ≤ 18 AND consent == "yes": [2]
 (accept: if $12 \leq \text{age} \leq 18$ AND consent == "yes" [2])
 (give 1 mark if condition is partially correct)

(b) Pseudocode:

```

SET Age TO 0 [1]
SET Consent TO ""
OUTPUT "Enter your age:"
INPUT Age [1]
OUTPUT "Do you have parental consent? (yes/no)"
INPUT Consent [1]
IF Age  $\geq 12$  AND Age  $\leq 18$  AND Consent == "yes" THEN
    OUTPUT "Registration successful" [1]
ELSE
    OUTPUT "You cannot register" [1]
ENDIF

```

END OF WEEKS 1-6

CUMULATIVE VOCABULARY LIST (WEEKS 1-6)

English	Chinese	Pinyin
algorithm	算法	suàn fǎ
pseudocode	伪代码	wěi dài mǎ
decomposition	分解	fēn jiě
input	输入	shū rù
process	处理	chǔ lǐ
output	输出	shū chū
sequence	顺序	shùn xù
variable	变量	biàn liàng
flowchart	流程图	liú chéng tú
terminal	起止框	qǐ zhǐ kuāng
process box	处理框	chǔ lǐ kuāng
decision diamond	判断菱形	pàn duàn líng xíng
selection	选择结构	xuǎn zé jié gòu
linear search	线性搜索	xiàn xìng sōu suǒ
binary search	二分搜索	èr fēn sōu suǒ
sorted	已排序	yǐ pái xù
trace table	追踪表	zhuī zōng biǎo
string (str)	字符串	zì fú chuàn
integer (int)	整数	zhěng shù
float	浮点数	fú diǎn shù
boolean (bool)	布尔值	bù ěr zhí
data type	数据类型	shù jù lèi xíng
type()	类型检查	lèi xíng jiǎn chá
input()	输入函数	shū rù hán shù
f-string	格式化字符串	gé shì huà zì fú chuàn
type conversion	类型转换	lèi xíng zhuǎn huàn
concatenate	连接	lián jiē
arithmetic operator	算术运算符	suàn shù yùn suàn fú
integer division	整数除法	zhěng shù chú fǎ
modulo	取模/取余	qǔ mó / qǔ yú
exponent	幂运算	mì yùn suàn
BODMAS	运算顺序	yùn suàn shùn xù

comparison operator 比较运算符 bǐ jiào yùn suàn fú

logical operator 逻辑运算符 luó jí yùn suàn fú

truth table 真值表 zhēn zhí biǎo

AND 并且 bìng qiě

OR 或者 huò zhě

NOT 非 fēi

if statement 条件语句 tiáo jiàn yǔ jù

else 否则 fǒu zé

elif 否则如果 fǒu zé rú guǒ

indentation 缩进 suō jìn

nested if 嵌套条件 qiàn tào tiáo jiàn

complex condition 复合条件 fù hé tiáo jiàn

debug 调试 tiáo shì

syntax error 语法错误 yǔ fǎ cuò wù

logic error 逻辑错误 luó jí cuò wù

DOCUMENT INFORMATION

Curriculum: VPA IGCSE Computer Science (0478) Year 9 Semester 2

Weeks: 1–6 of 12

Topics: Algorithms, Flowcharts, Searching, Python Fundamentals, Operators,
Selection

Written for: EAL learners, mixed ability, 40-minute lessons

IDE: Thonny (Python 3.10+)

Last Updated: June 2025

Next: Weeks 7–12 (Iteration, Arrays, String Handling, File Handling, Revision)